[ADS5034-01] Computer Vision

12. Classification (2)

Jae-Young Choi

Sungkyunkwan University

Outline



- Classification
  - Image classification
- ConvNet (CNN)
  - Convolutional Networks
  - Neural Networks
- AlexNet / ResNet
- Vision Transformer

Classification

Image Classification

- **Goal:** Given an image, we want to identify what class that image belongs to

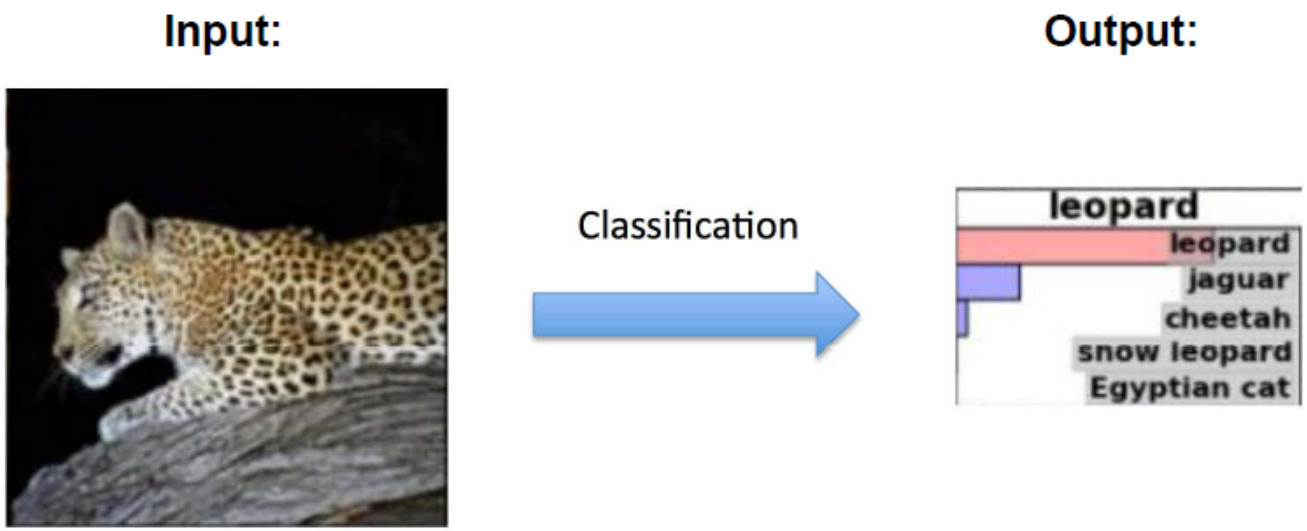


Image Classification

- **Goal:** Given an image, we want to identify what class that image belongs to

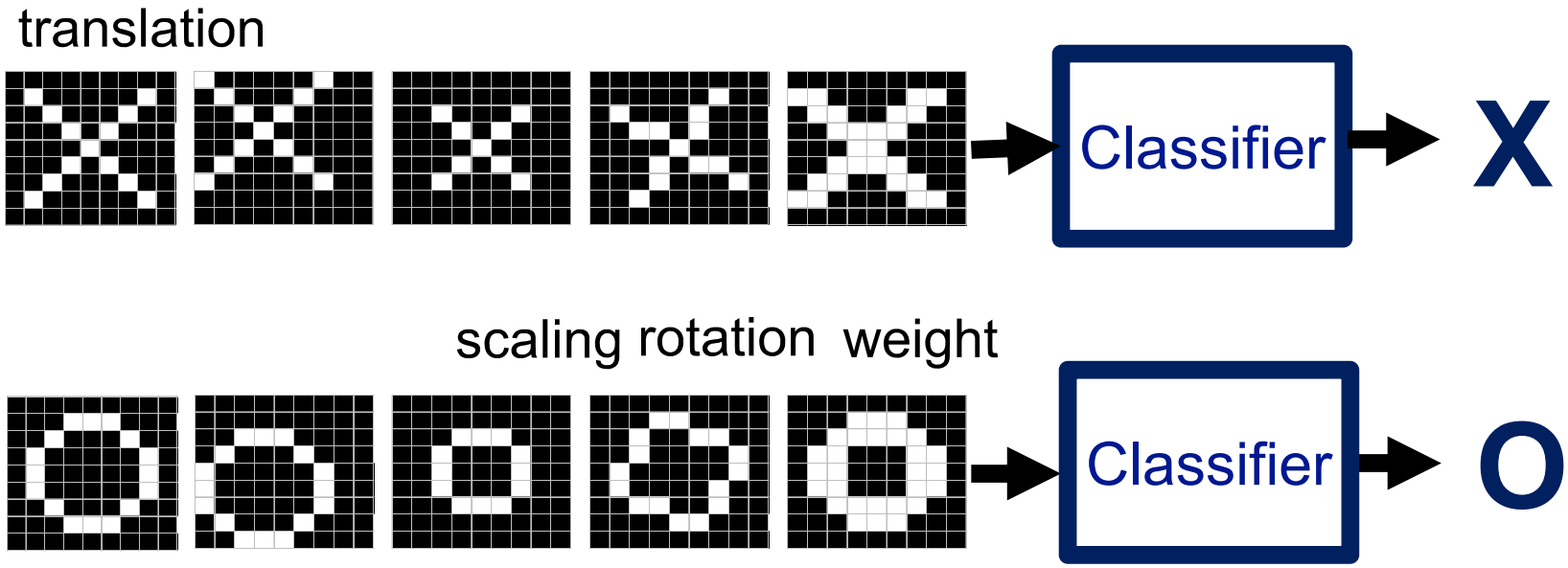


Image Classification

- Recall: image classification pipeline

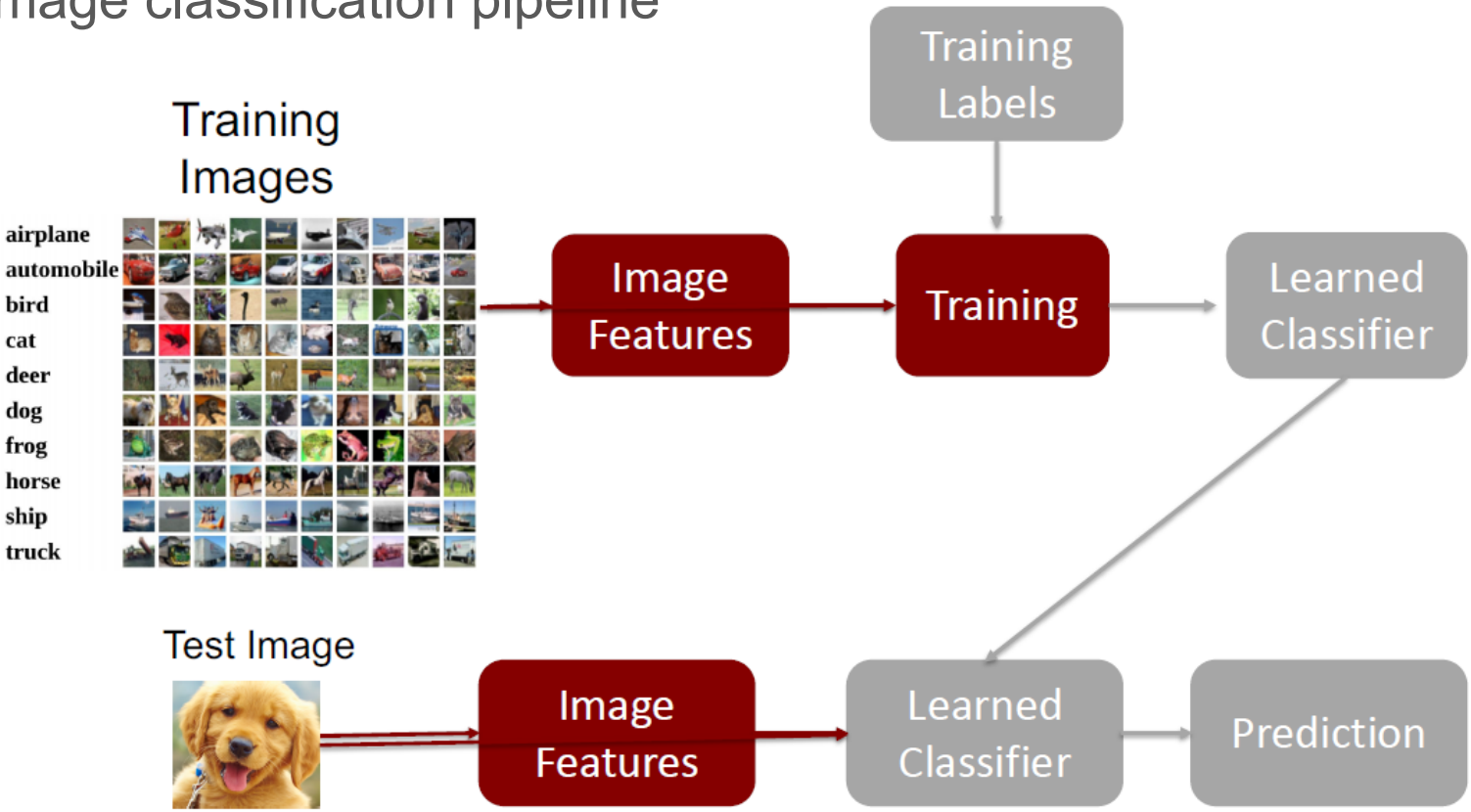
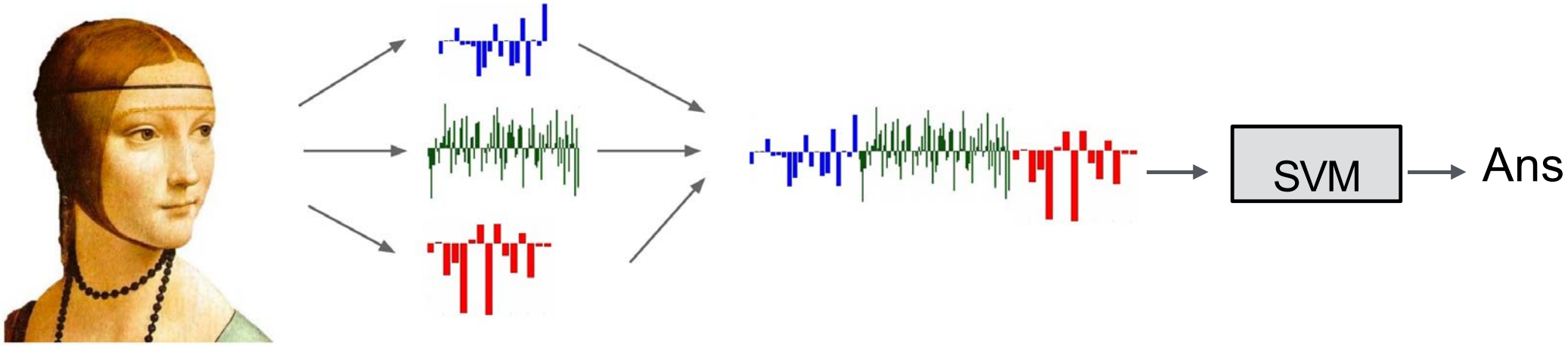


Image Classification

- Life Before Deep Learning



Input Pixels

Extract Hand-Crafted Features

Concatenate into a vector ***x***

Linear Classifier

Image Classification

- Bag of Visual Words

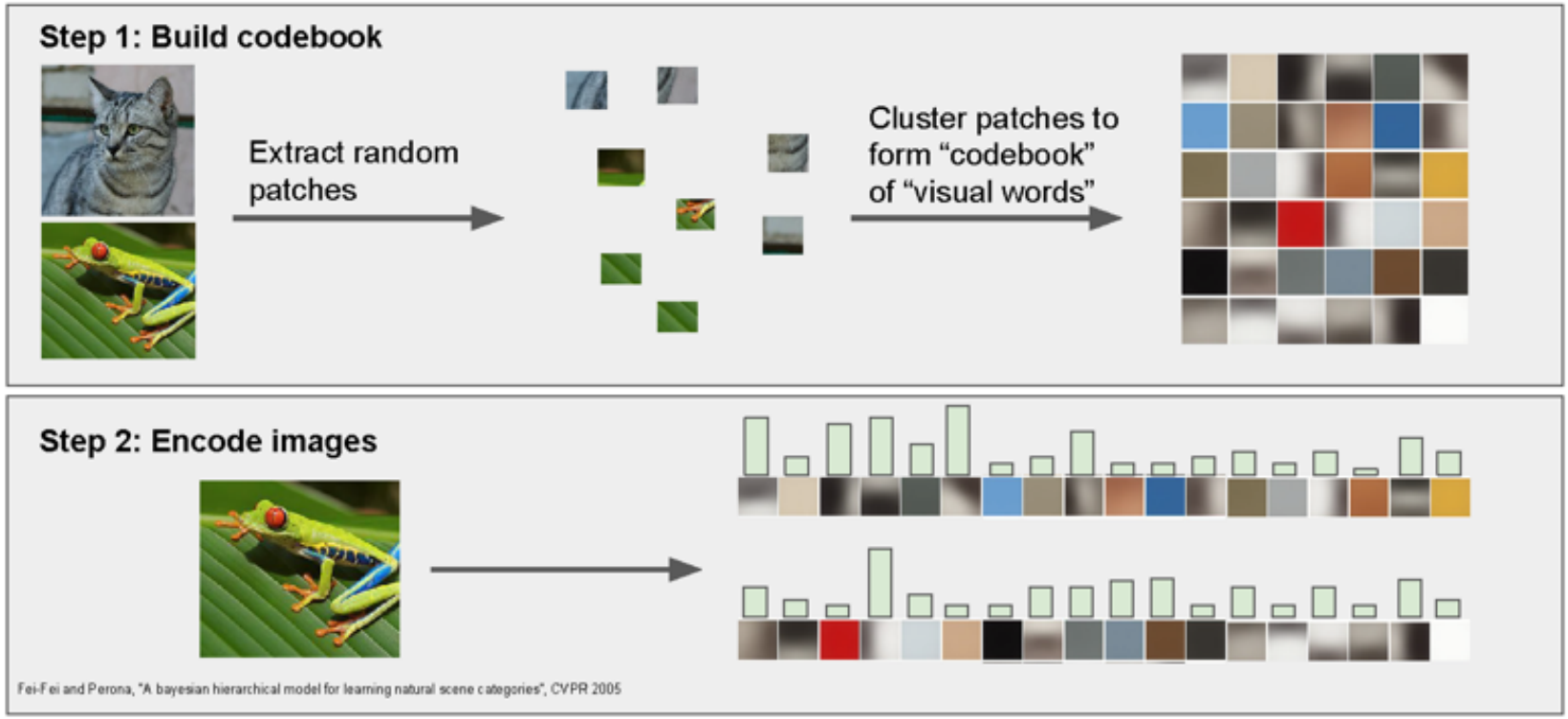


Image Classification

- Let's change the features by adding another layer

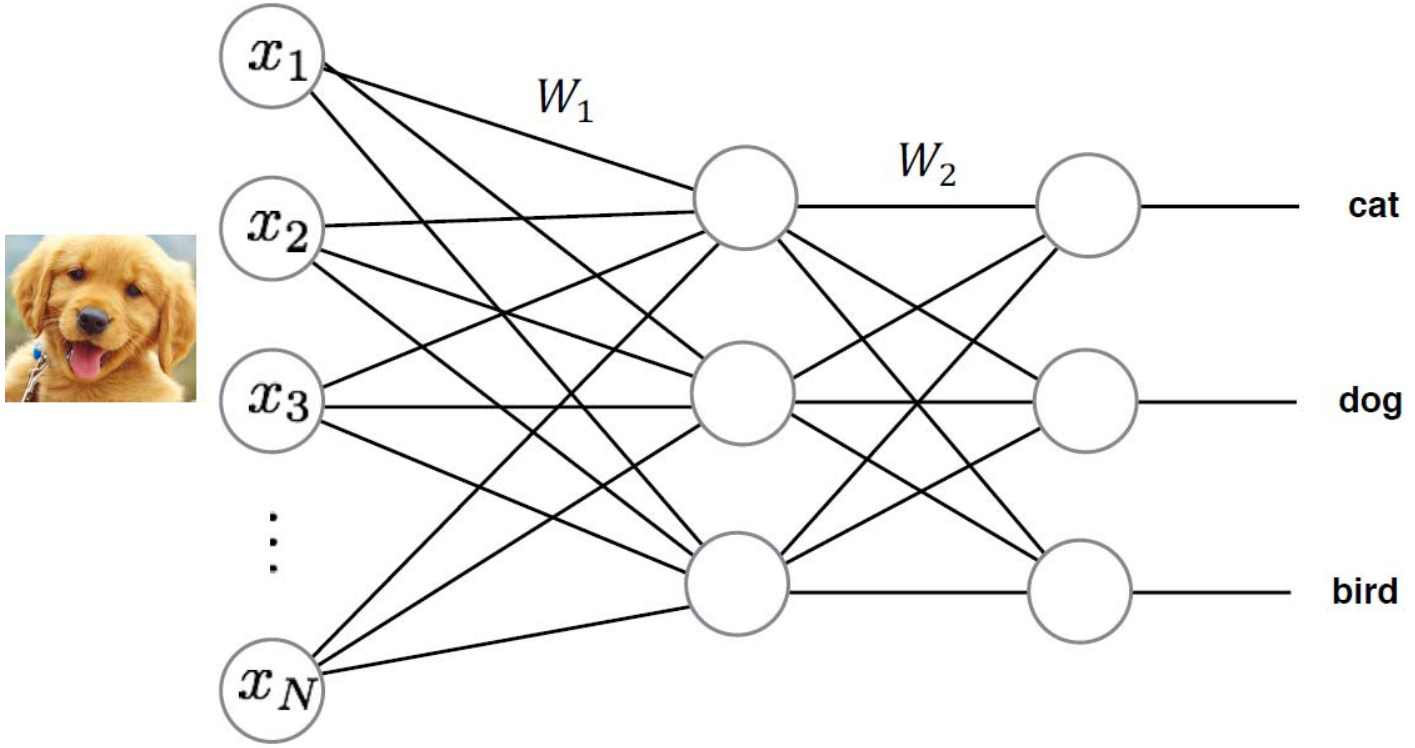


Image Classification

- We can make feature of images into a vector,
- But these representations lose spatial information



Raw pixels
Raw pixels + (x,y)
PCA
LDA
BoW
BoW + spatial pyramids

Image Vector



Can we design a neural network to retain spatial information?

Image Classification

- YES, We have already designed convolutions to be spatially invariant

We can apply 3 filters to operate over all three color channels

Let's represent this new kernel as a tensor as well

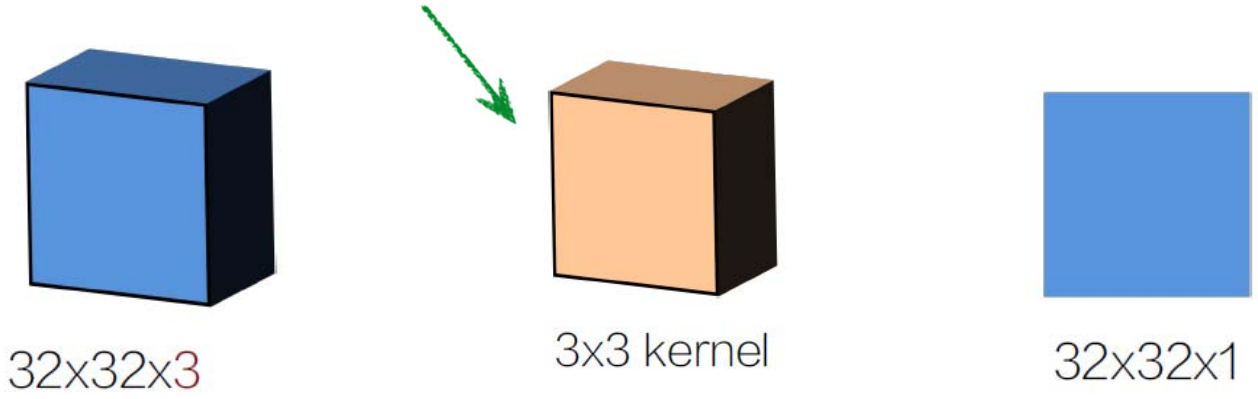


Image Classification

- The convolution layer

We can apply K = 64 multiple kernels in parallel to learn different kinds of features

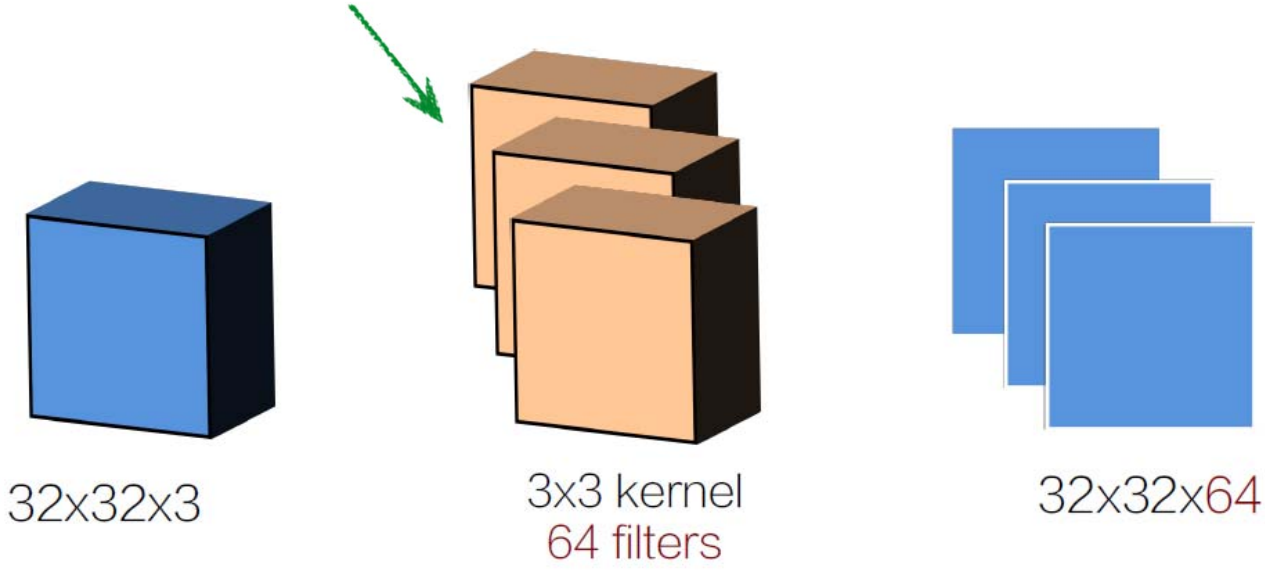


Image Classification

- The convolution layer
  - How many weights would a linear layer need to convert the input to the following feature?

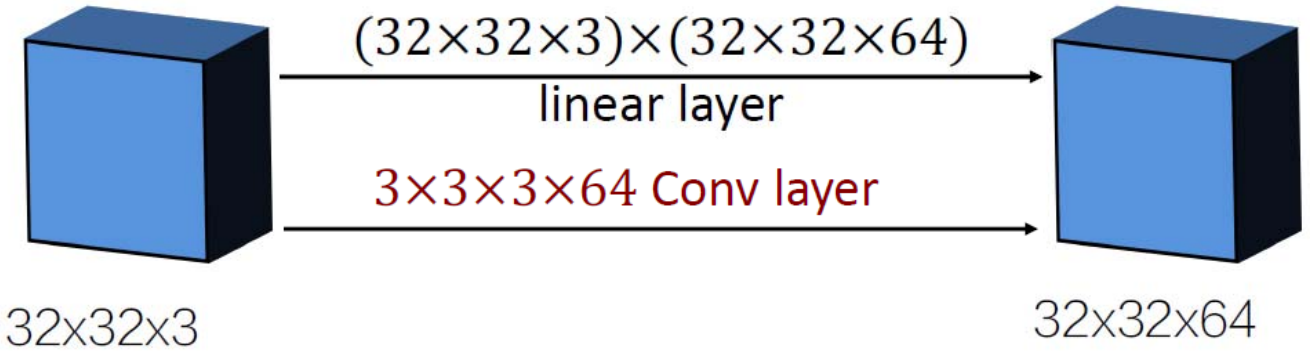
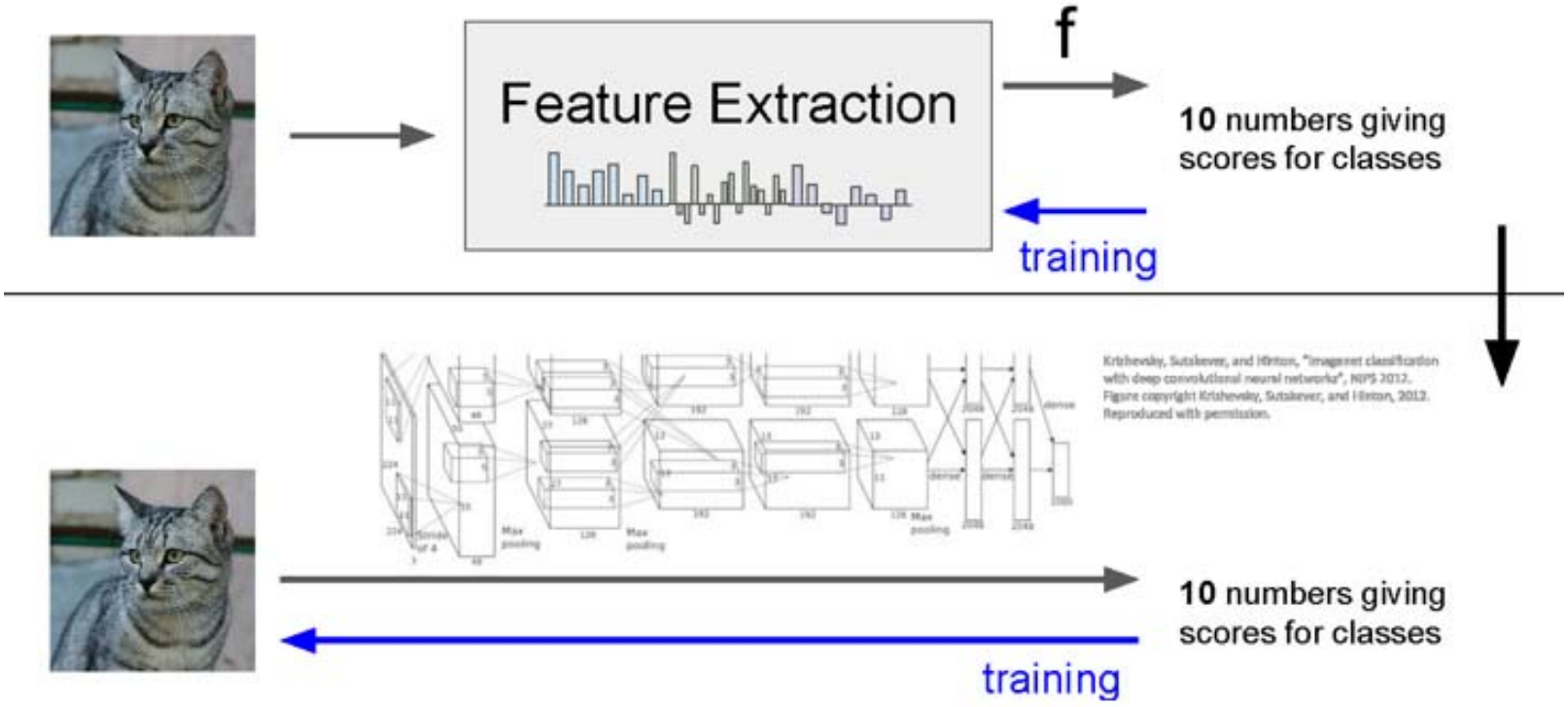


Image Classification

- Image Features vs ConvNets



<ConvNet>

Convolutional Networks

Neural Networks

Convolutional Neural Networks

Convolutional ~~Neural~~ Networks

- 2D convolution
  - f : the values in a 2D grid that we want to convolve
  - g : convolutional weights of size MxN

$$h = f \otimes g$$

$$h_{ij} = \sum_{m=0}^{M} \sum_{n=0}^{N} f(i-m, j-n) g(m,n)$$

A sliding window operation across the entire grid f

Convolutional ~~Neural~~ Networks

- 2D convolution
  - example

$f =$ 

ConvNets aim to learn convolutional weights directly from the data!!

$g_1 =$

0	0	0
0	1	0
0	0	0

$g_2 =$

0.107	0.113	0.107
0.113	0.119	0.113
0.107	0.113	0.107

$g_3 =$

-1	0	1
-2	0	2
-1	0	1

$f \otimes g_1$



Unchanged Image

$f \otimes g_2$



Blurred Image

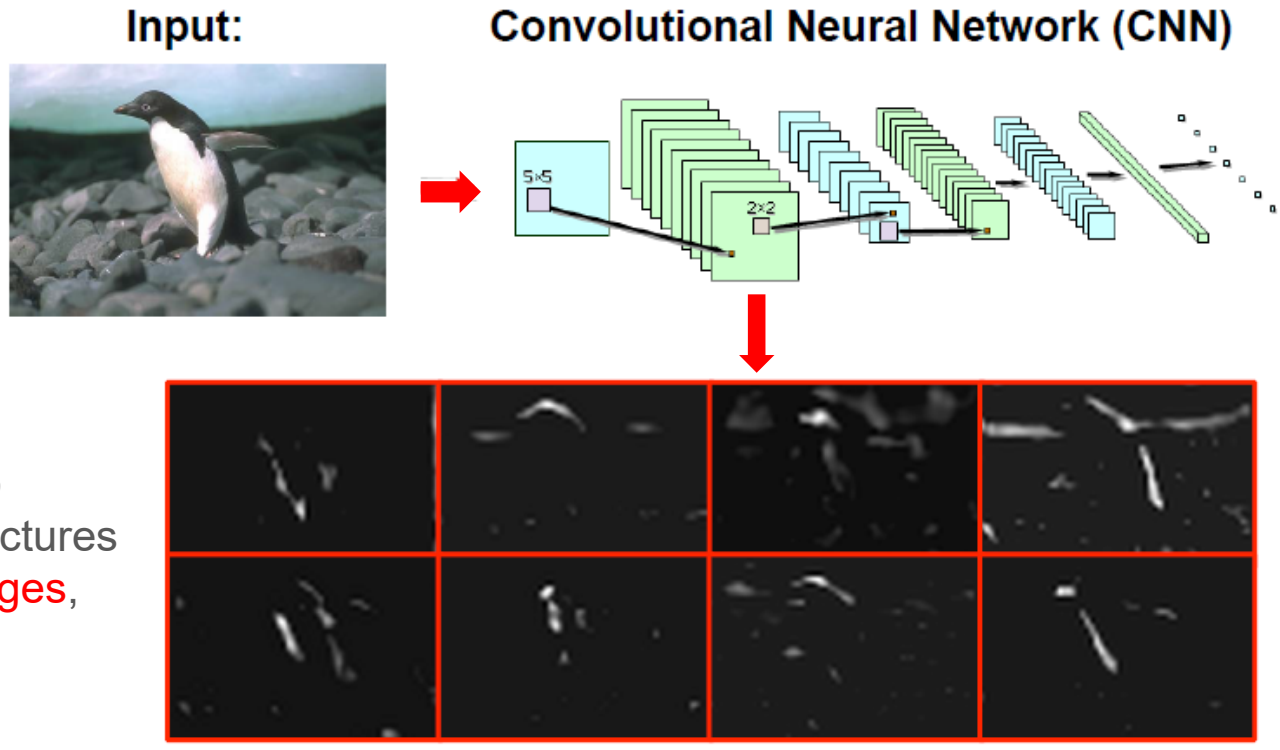
$f \otimes g_3$



Vertical Edges

Convolutional ~~Neural~~ Networks

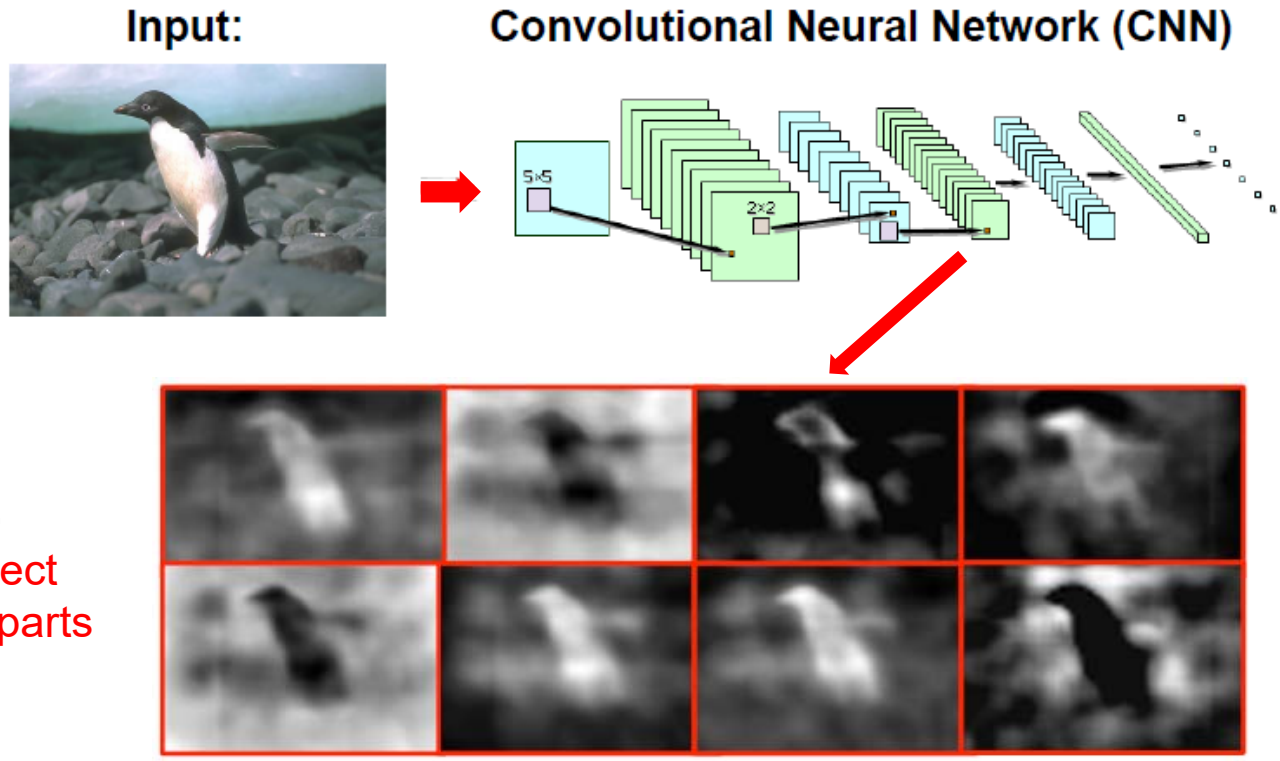
- 2D convolution
  - example



Early layers learn to detect low level structures such as oriented edges, colors and corners

Convolutional ~~Neural~~ Networks

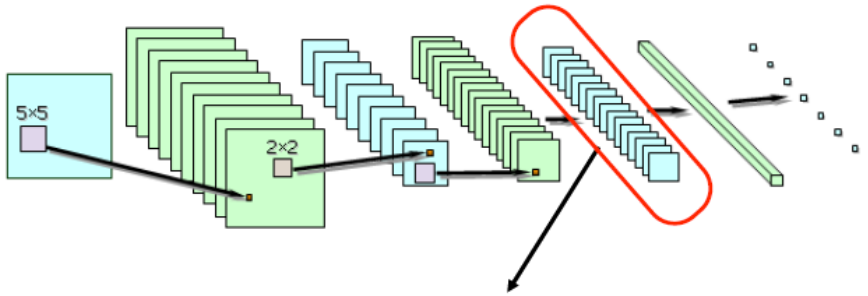
- 2D convolution
  - example



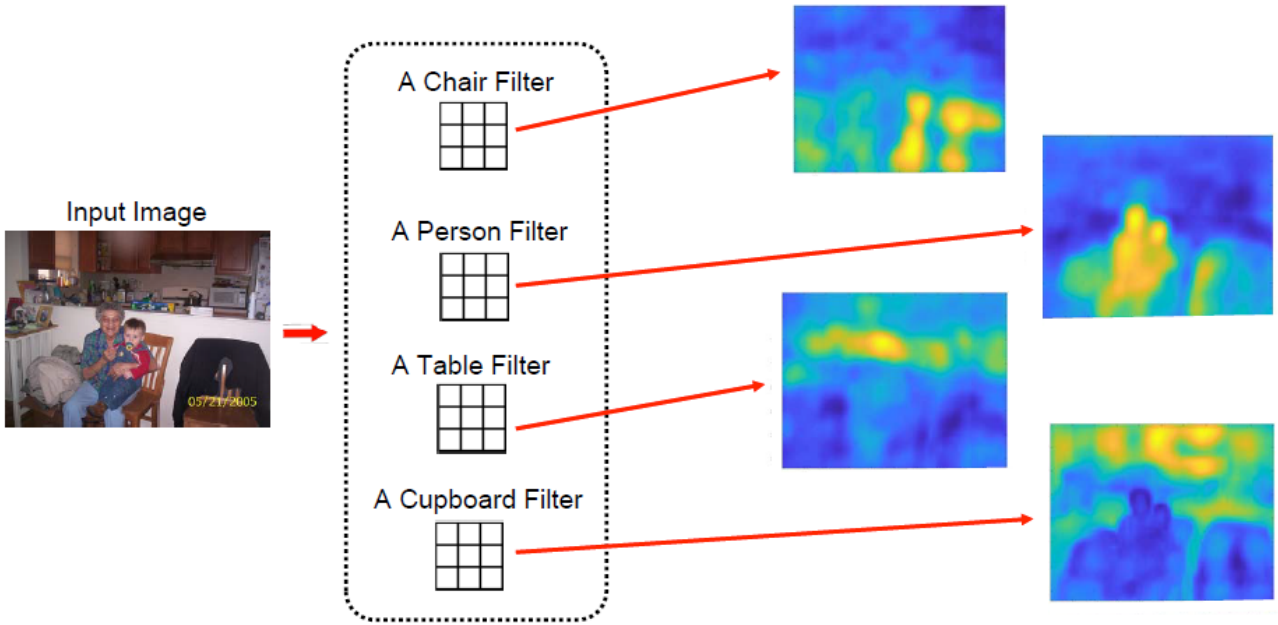
Deep layers learn to detect high-level object structures and their parts

Convolutional ~~Neural~~ Networks

- 2D convolution
  - example

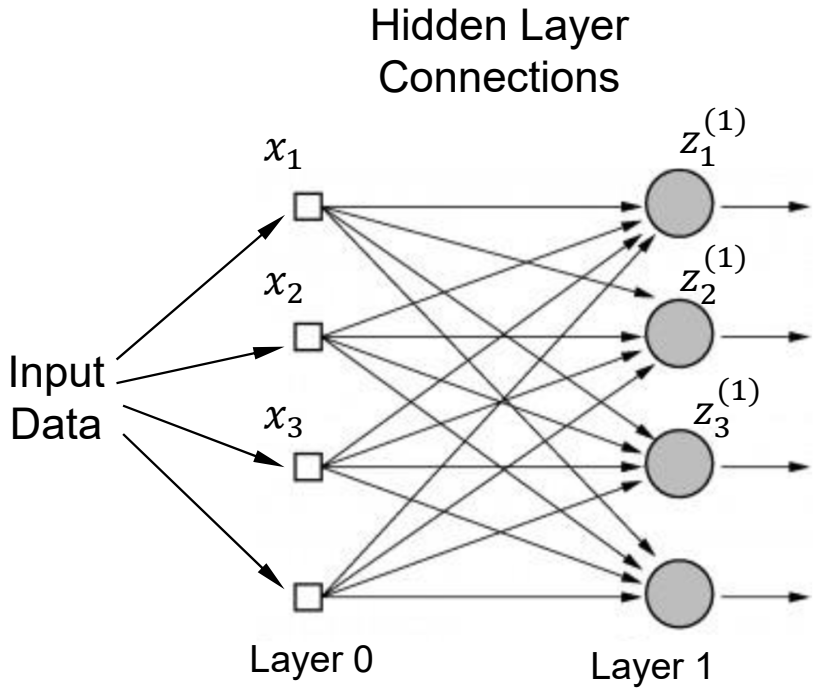


A Closer Look inside the Convolutional Layer



~~Convolutional~~ Neural Networks

- Fully Connected Layers



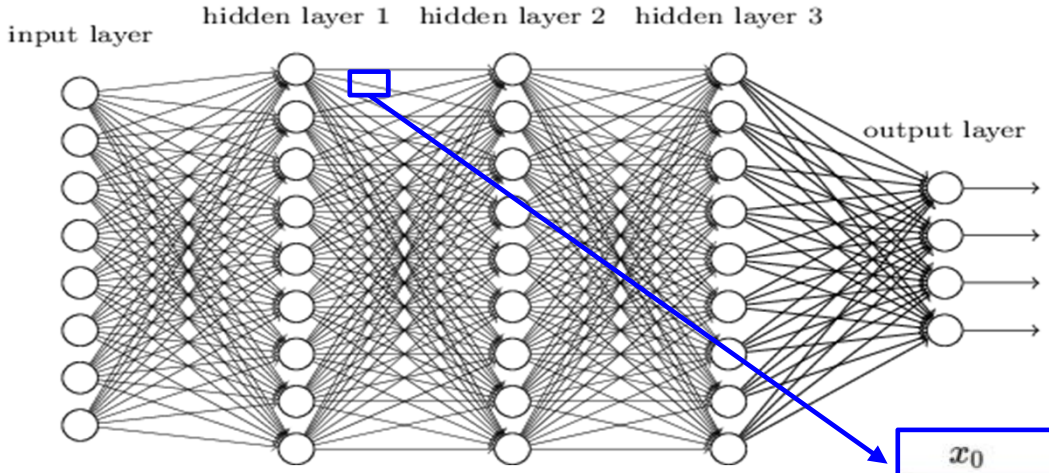
$z_i^{(l)}$ - the output unit i in layer l

$W_{ij}^{(l)}$ - the weight connection between unit j in layer l and unit i in layer $l+1$

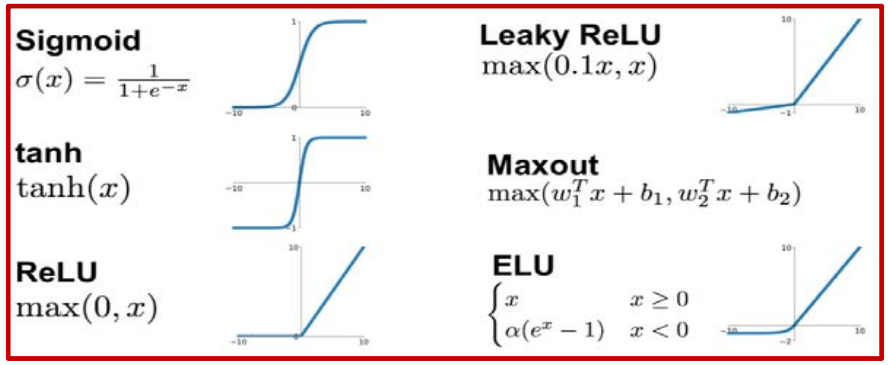
$$z_1^{(1)} = W_{11}^{(0)} x_1 + W_{12}^{(0)} x_2 + W_{13}^{(0)} x_3 \cdots$$
$$z_2^{(1)} = W_{21}^{(0)} x_1 + W_{22}^{(0)} x_2 + W_{23}^{(0)} x_3 \cdots$$
$$z_3^{(1)} = W_{31}^{(0)} x_1 + W_{32}^{(0)} x_2 + W_{33}^{(0)} x_3 \cdots$$
$$\vdots$$

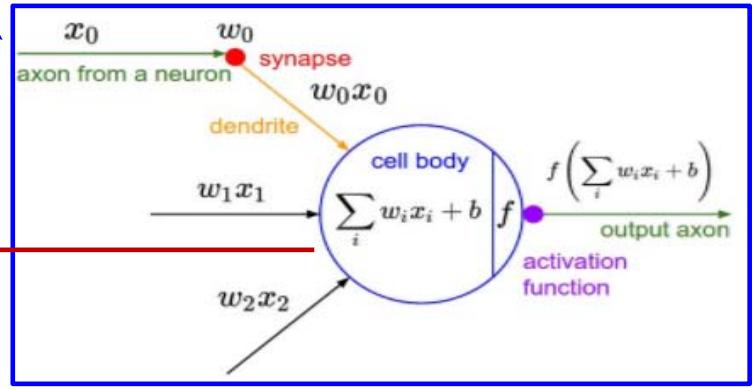
~~Convolutional~~ Neural Networks

- An artificial neural network which consists of at least 3 layers



Deep networks typically have many layers and potentially millions of parameters



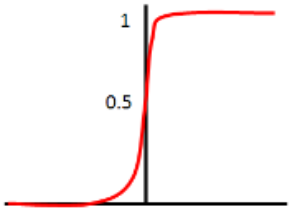


~~Convolutional~~ Neural Networks

- Activation Function (Non-linear function)

✓ Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



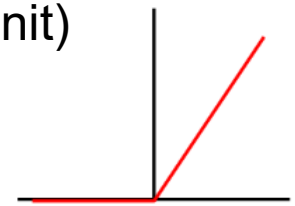
✓ tanh

$$\tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$$



✓ ReLu (Rectified Linear Unit)

$$\text{ReLU} = \begin{cases} x, & if\ x \geq 0 \\ 0, & if\ x < 0 \end{cases}$$



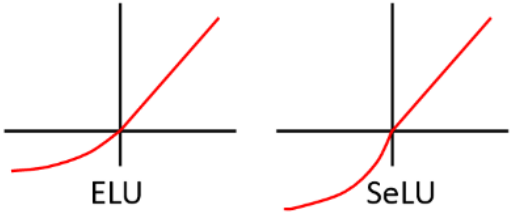
✓ Leaky ReLu

$$\text{Leaky ReLU} = \begin{cases} x, & if\ x \geq 0 \\ 0.01x, & if\ x < 0 \end{cases}$$



✓ ELU (Exponential Linear Unit)

$$f(\alpha,x) = \begin{cases} x & if\ x > 0 \\ \alpha(e^{x}-1) & if\ x \leq 0 \end{cases}$$



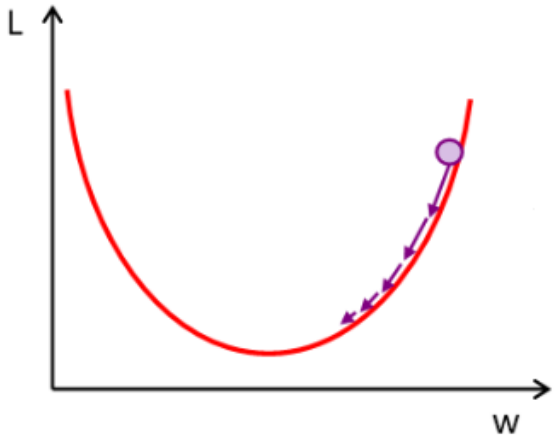
~~Convolutional~~ Neural Networks

- Cost function / Loss function
  - **Cost function** is usually more general. It might be a sum of loss functions over your training set plus some model complexity penalty (regularization)
    - Mean Squared Error: $MSE(\theta) = \frac{1}{N}\sum_{i=1}^{N}(f(x_i|\theta) - y_i)^2$
    - SVM cost function: $SVM(\theta) = \|\theta\|^2 + C\sum_{i=1}^{N}\xi_i$ (there are additional constraints connecting ξ_i with C and with training set)
  - **Loss function** is usually a function defined on a data point, prediction and label, and measures the penalty
    - Square loss: $l(f(x_i|\theta), y_i) = (f(x_i|\theta) - y_i)^2$, used in linear regression
    - Hinge loss: $l(f(x_i|\theta), y_i) = \max(0, 1 - f(x_i|\theta)y_i)$, used in SVM
    - 0/1 loss: $l(f(x_i|\theta), y_i) = 1 \iff f(x_i|\theta) \neq y_i$, used in theoretical analysis and definition of accuracy

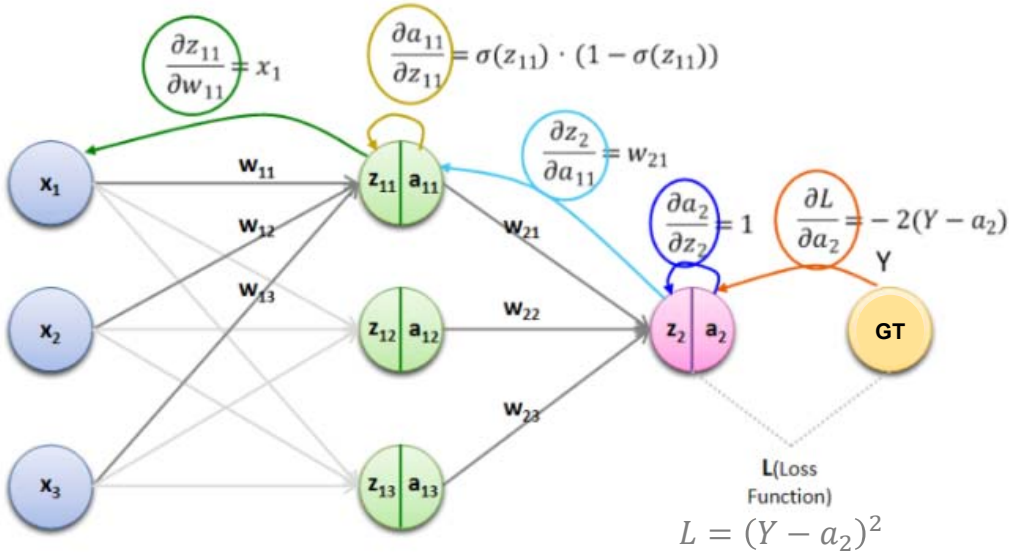
~~Convolutional~~ Neural Networks

- Gradient Descent
  - Gradient descent is an optimization algorithm that's used when training a machine learning model.
  - Gradient descent in machine learning is simply used to find the values of a function's parameters (coefficients) that minimize a cost function as far as possible.

$$W_{new} = W_{old} - \gamma \nabla_W L$$

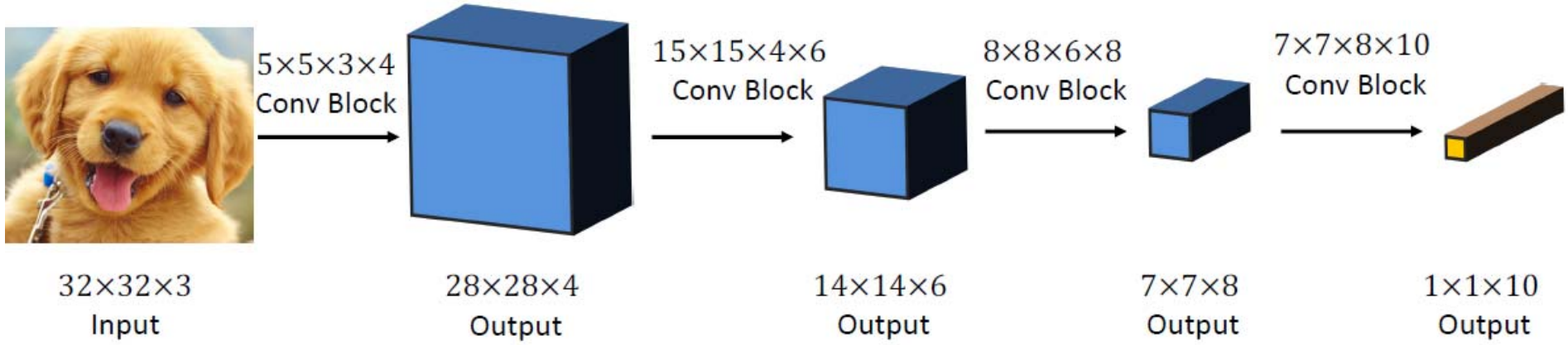


$$\frac{\partial L}{\partial w_{11}} = ????$$



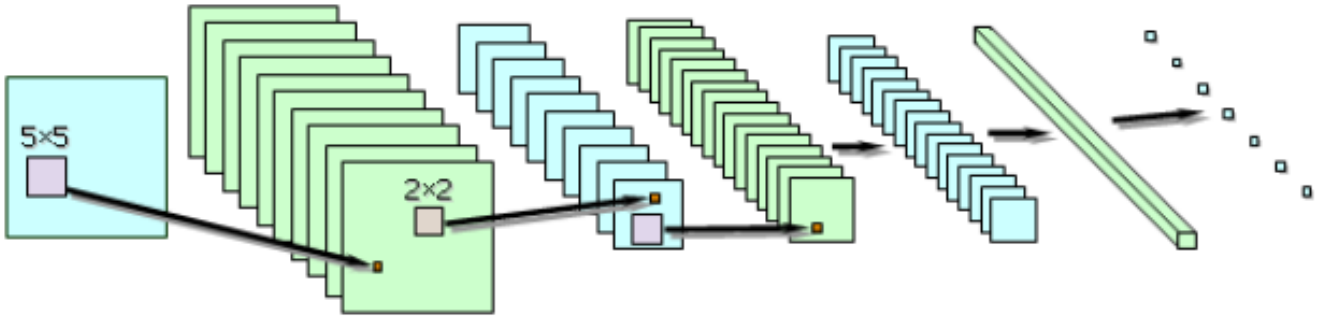
ConvNets

- A convolutional neural network architecture



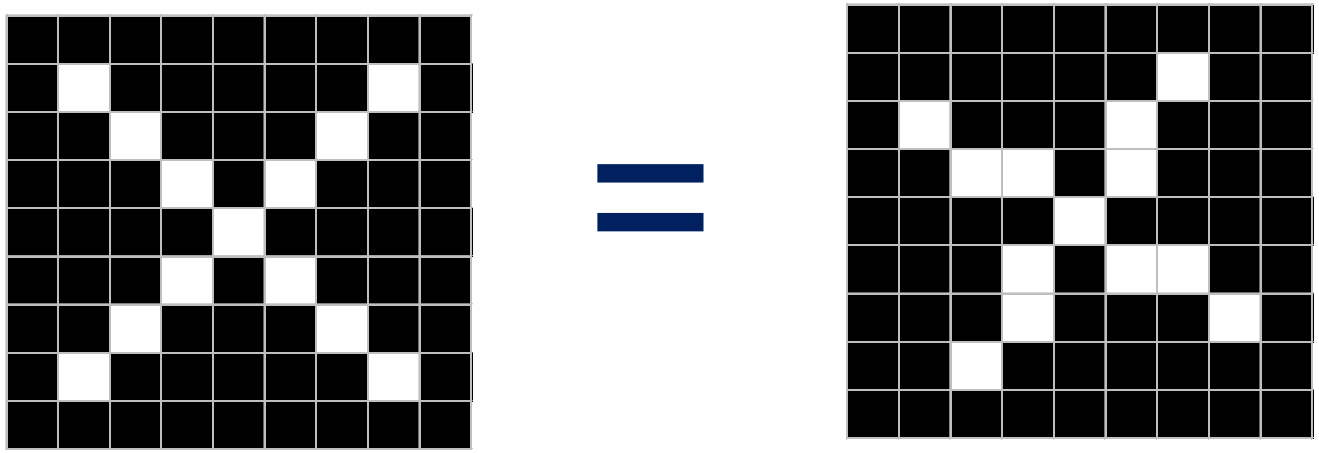
ConvNets

- How the key operations are encoded in standard CNNs:
  - Convolutional Layers: 2D Convolution
  - Activation Layers: Activation function
  - Pooling Layers: Subsampling
  - Fully Connected Layers: Matrix Multiplication



ConvNets : Convolution

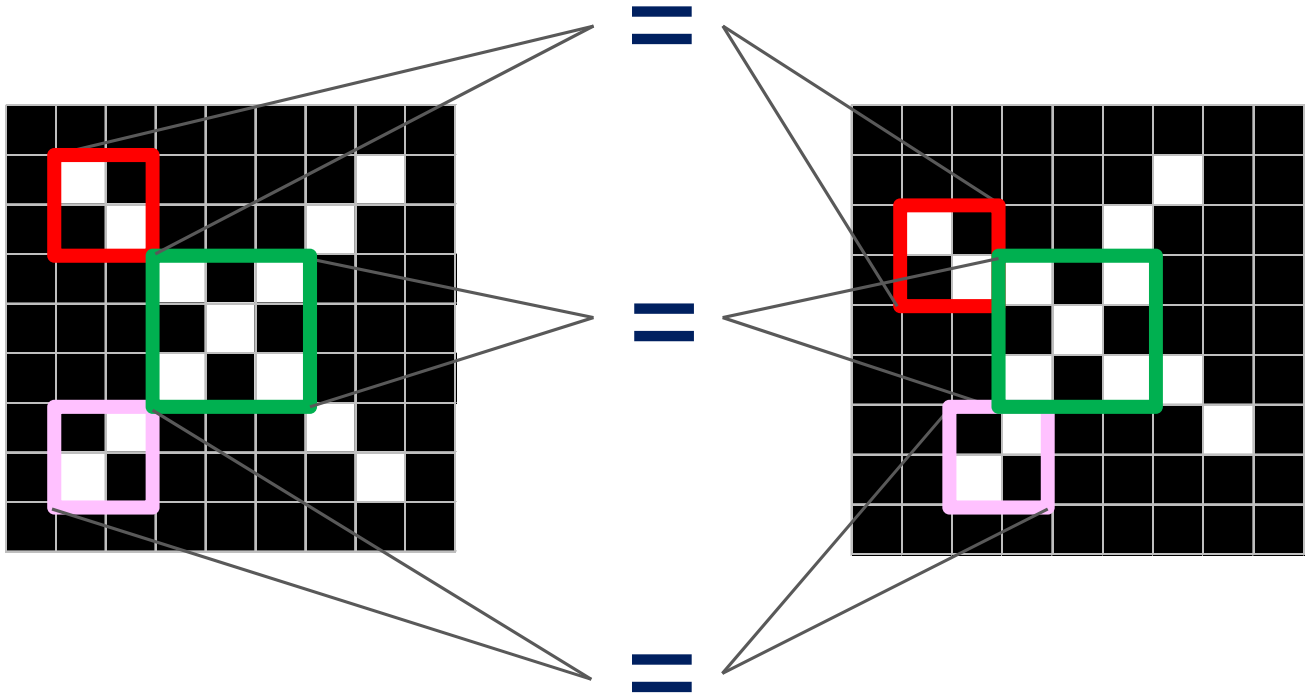
- Convolution
  - A way to find out local features



How to determine both are the same?

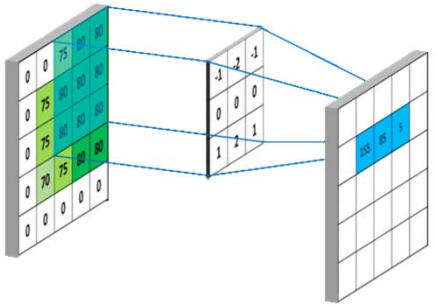
ConvNets : Convolution

- Both are partially matching
  - Critical LOCAL features are the same



How about finding local features, and Matching them ?

ConvNets : Convolution



- Convolution
  - A way to find out local features

$$I * K = \sum K_{ij} \times I_{ij}$$

1	0	1
0	1	1
1	1	0

*

1	0	1
0	1	0
1	0	1

= 4

ConvNets : Convolution

-1	-1	1
-1	1	-1
1	-1	-1

$$(I * K)_{xy} = \sum_{i=1}^{w} \sum_{j=1}^{w} K_{ij} \cdot I_{x+i-1,y+j-1}$$

0	0	0	0	0	0	0	0
0	0	0	1	1	0	0	0
0	0	1	0	1	1	0	0
0	1	0	0	0	0	1	0
0	1	0	0	0	0	1	0
0	0	1	0	1	1	0	0
0	0	0	1	1	0	0	0
0	0	0	0	0	0	0	0

	-1						

ConvNets : Convolution

- Convolution Stride
  - During convolution, the weight ‘slide’ along the input the generate each output

-1	-1	1
-1	1	-1
1	-1	-1

0	0	0	0	0	0	0	0
0	0	0	1	1	0	0	0
0	0	1	0	1	1	0	0
0	1	0	0	0	0	1	0
0	1	0	0	0	0	1	0
0	0	1	0	1	1	0	0
0	0	0	1	1	0	0	0
0	0	0	0	0	0	0	0

	-1						
	-2						

ConvNets : Convolution

-1	-1	1
-1	1	-1
1	-1	-1

0	0	0	0	0	0	0	0
0	0	0	1	1	0	0	0
0	0	1	0	1	1	0	0
0	1	0	0	0	0	1	0
0	1	0	0	0	0	1	0
0	0	1	0	1	1	0	0
0	0	0	1	1	0	0	0
0	0	0	0	0	0	0	0

	-1						
	-2						
	1						

ConvNets : Convolution

-1	-1	1
-1	1	-1
1	-1	-1

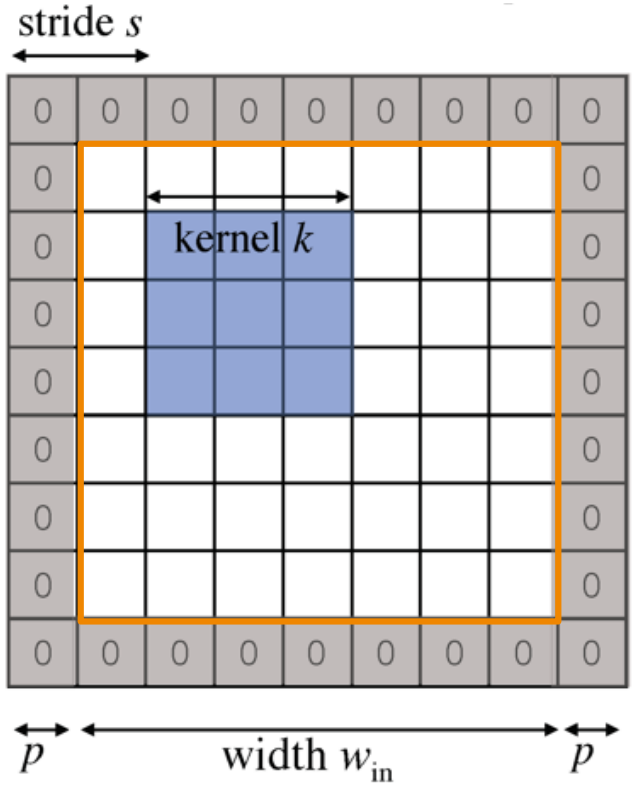
$$(I * K)_{xy} = \sum_{i=1}^{w} \sum_{j=1}^{w} K_{ij} \cdot I_{x+i-1,y+j-1}$$

0	0	0	0	0	0	0	0
0	0	0	1	1	0	0	0
0	0	1	0	1	1	0	0
0	1	0	0	0	0	1	0
0	1	0	0	0	0	1	0
0	0	1	0	1	1	0	0
0	0	0	1	1	0	0	0
0	0	0	0	0	0	0	0

	-1	-2	0	-2	-1	1	
	-2	3	-2	-2	-2	-2	
	1	-1	-2	0	-4	-1	
	-1	-3	0	-2	0	1	
	-2	-1	-4	0	2	-2	
	1	-2	0	0	-3	-1	

ConvNets : Convolution

- How big is the output?



In general, the output has size:

$$w_{out} = \left\lfloor \frac{w_{in} + 2p - k}{s} \right\rfloor + 1$$

e.g. w_{in} = 7, k = 3:
s=1, p=0 $\Rightarrow$ $w_{out} = (7-3)/1 + 1 = 5$
s=1, p=1 $\Rightarrow$ $w_{out} = (7+2-3)/1 + 1 = 7$!!

ConvNets : Convolution

- Adding the Pad
  - In practice : Common to zero pad the border

e.g. w_{in} = 8, k = 3:
s=1, p=0 $\Rightarrow w_{\text{out}} = (8-3)/1 + 1 = 6$
s=1, p=1 $\Rightarrow w_{\text{out}} = (8+2-3)/1 + 1 = 8$

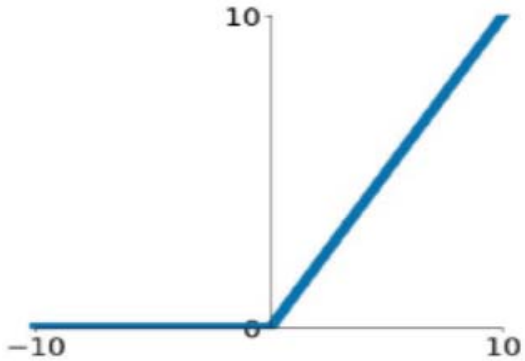
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	1	0	1	1	0	0	0
0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	1	0	0
0	0	0	1	0	1	1	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

-1	-1	1
-1	1	-1
1	-1	-1

0	0	-1	-2	0	1	0	0
0	-1	-2	0	-2	-1	1	0
-1	-2	3	-2	-2	-2	-2	1
-2	1	-1	-2	0	-4	-1	0
0	-1	-3	0	-2	0	1	-2
1	-2	-1	-4	0	2	-2	-1
0	1	-2	0	0	-3	-1	0
0	0	1	0	-2	-1	0	0

ConvNets : Activation functions

- ReLU (Rectified Linear Unit)
  - $f(x) = \max(0, x)$
  - Does not saturate
  - Computationally efficient
  - Convergence much faster than sigmoid or tanh



- Problems
  - Not zero-centered output
  - Only keeps $x > 0$

ConvNets : Convolution

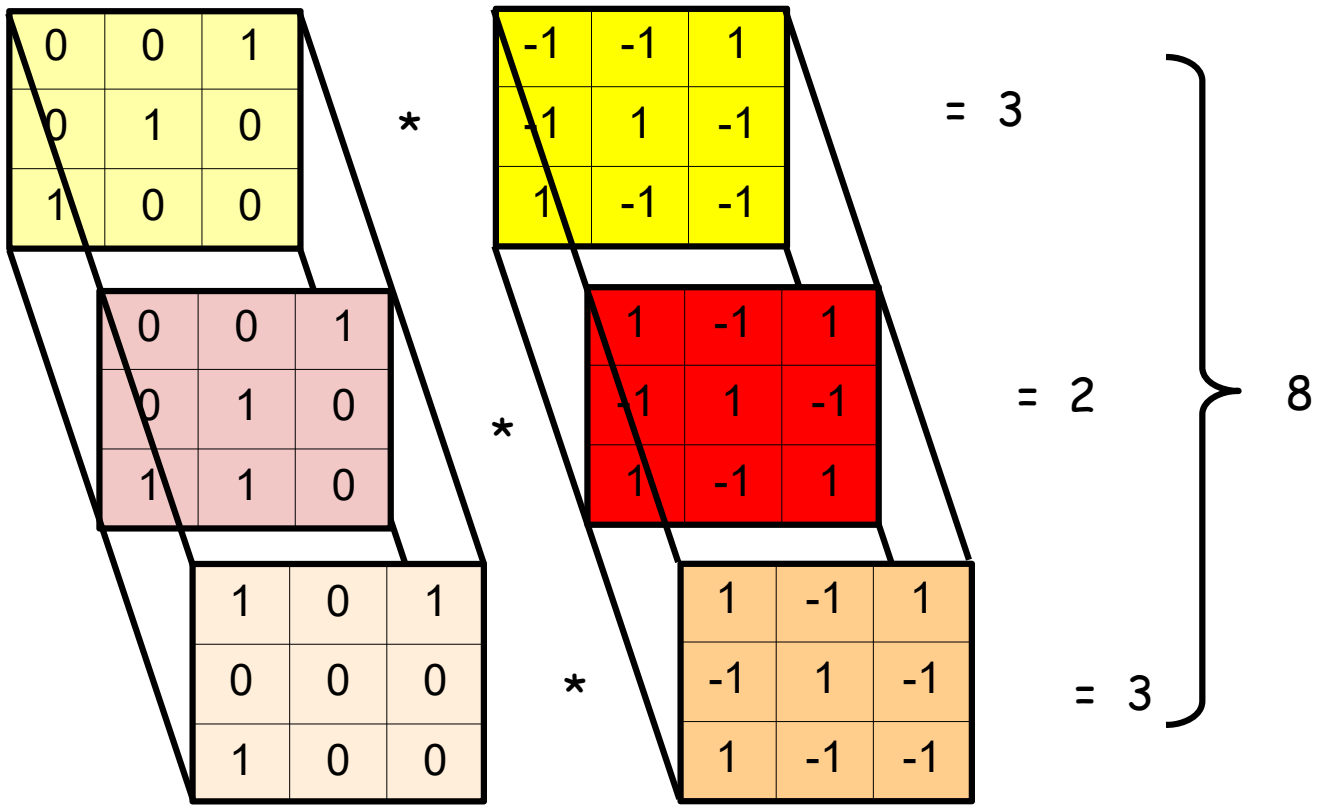
-1	-1	1
-1	1	-1
1	-1	-1

$$(I * K)_{xy} = \sum_{i=1}^{w} \sum_{j=1}^{w} K_{ij} \cdot I_{x+i-1,y+j-1}$$

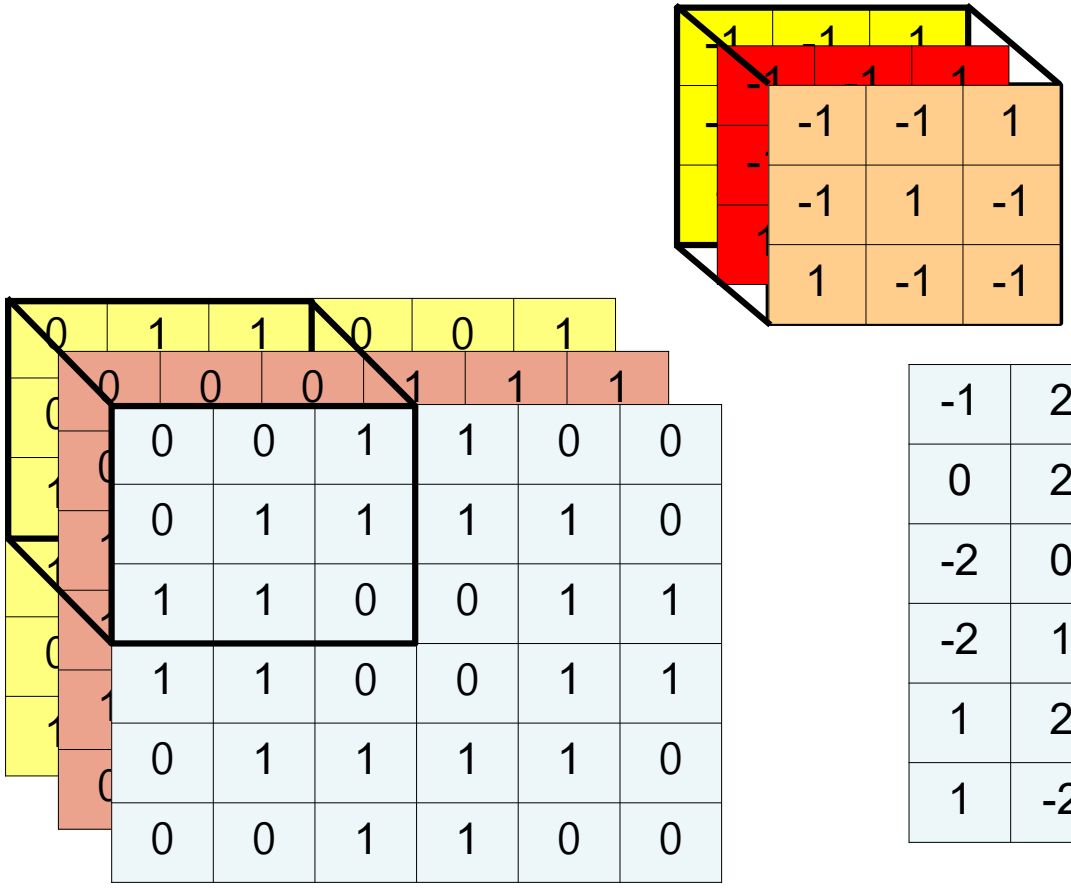
0	0	0	0	0	0	0	0
0	0	0	1	1	0	0	0
0	0	1	0	1	1	0	0
0	1	0	0	0	0	1	0
0	1	0	0	0	0	1	0
0	0	1	0	1	1	0	0
0	0	0	1	1	0	0	0
0	0	0	0	0	0	0	0

0	0	0	0	0	1	0	0
0	0	0	0	0	0	1	0
0	0	3	0	0	0	0	1
0	1	0	0	0	0	0	0
0	0	0	0	0	0	1	0
1	0	0	0	0	2	0	0
0	1	0	0	0	0	0	0
0	0	1	0	0	0	0	0

ConvNets : Convolution



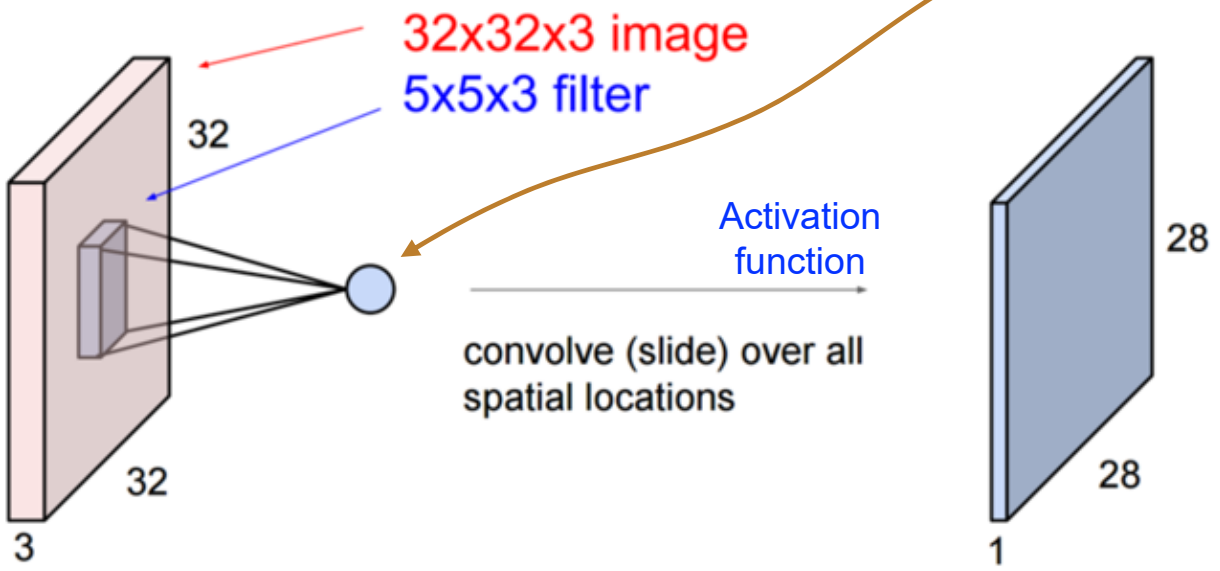
ConvNets : Convolution



-1	2	4	-2	-1	-4
0	2	5	1	3	2
-2	0	4	2	-6	1
-2	1	0	-2	3	-3
1	2	-4	4	2	-2
1	-2	-4	3	2	1

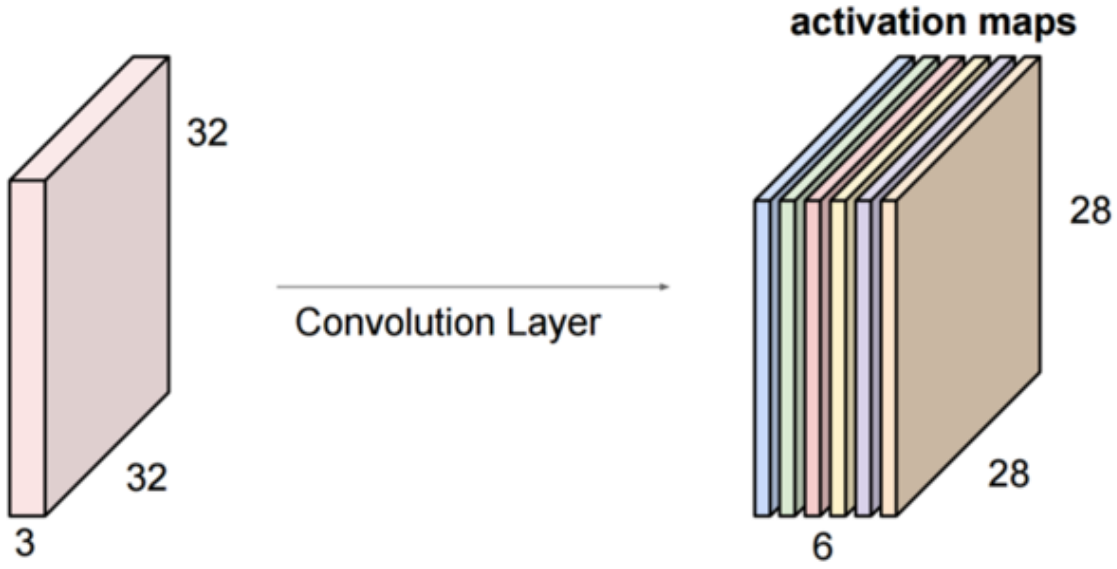
ConvNets : Convolution

- (Recapitulation)
  - The convolution result between the filter and a small 5x5x3 chunk of the image is a number (5x5x3 = 75-dimensional dot product + bias, $\omega^T x + b$)



ConvNets : Convolution

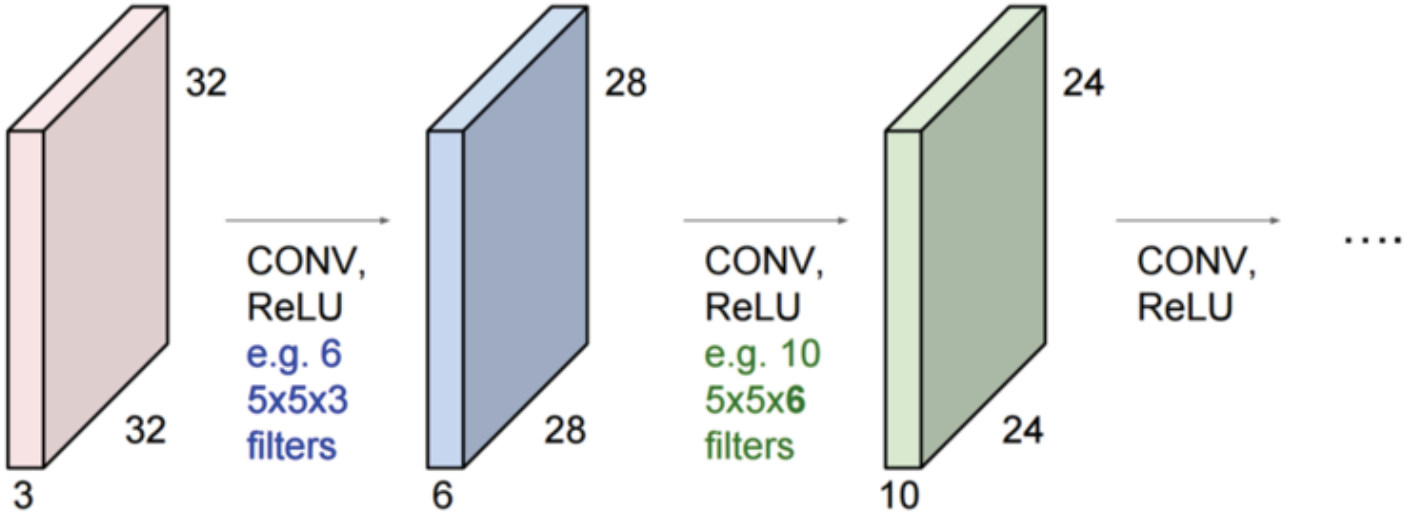
- (Recapitulation)
  - For example, if we had six 5x5 filters, we'll get 6 separate activation maps



We stack these up to get a 'new image' of size 28x28x6

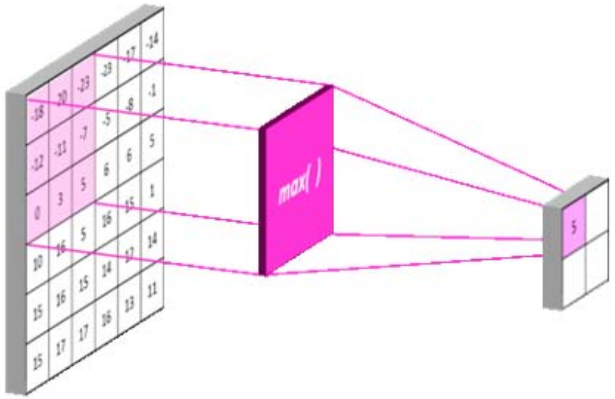
ConvNets : Convolution

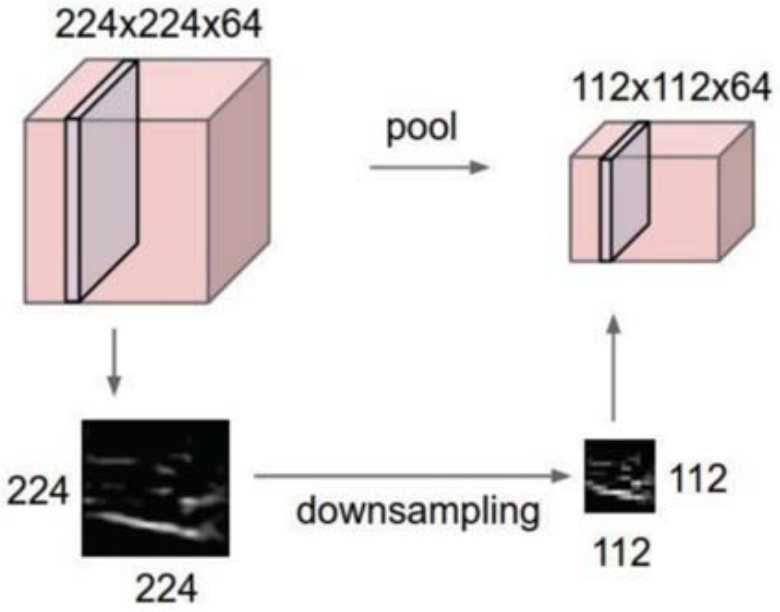
- (Recapitulation)
  - A ConvNet is a sequence of convolutional layers, interspersed with activation functions (and possibly other layer types)



ConvNets : Pooling

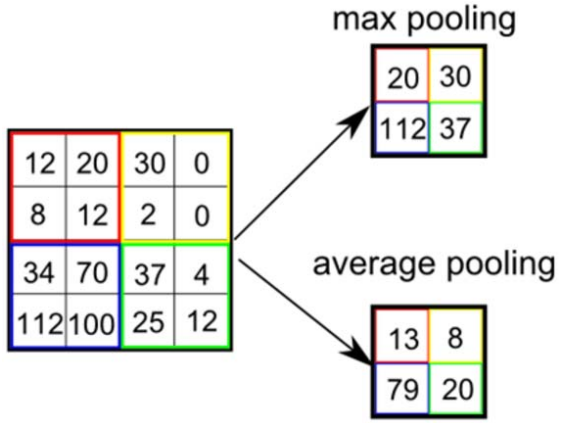
- Reduce the spatial size and computation power
- Makes the representations smaller and more manageable
- Operates over each activation map independently





ConvNets : Pooling

- Subsampling from *m* by *m* pixels into 1 pixels
- Max, averaging or L^p pooling



0	0	0	0	0	1	0	0
0	0	0	0	0	0	1	0
0	0	3	0	0	0	0	1
0	1	0	0	0	0	0	0
0	0	0	0	0	0	1	0
1	0	0	0	0	2	0	0
0	1	0	0	0	0	0	0
0	0	1	0	0	0	0	0

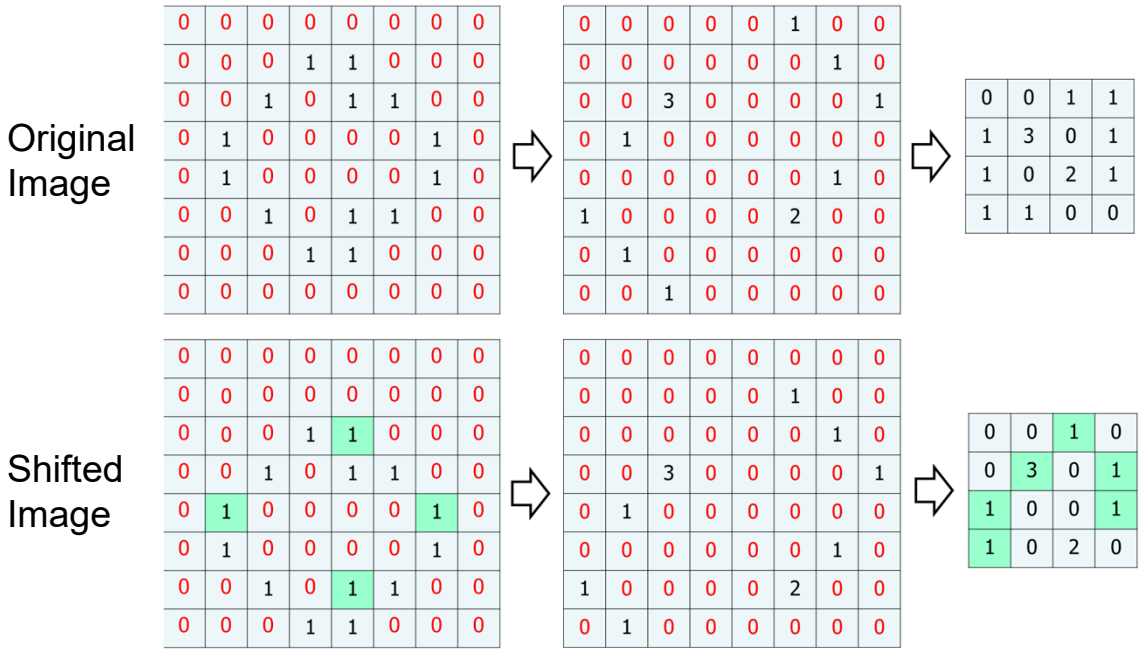
Feature map

0	0	1	1
1	3	0	1
1	0	2	1
1	1	0	0

Subsampled feature map

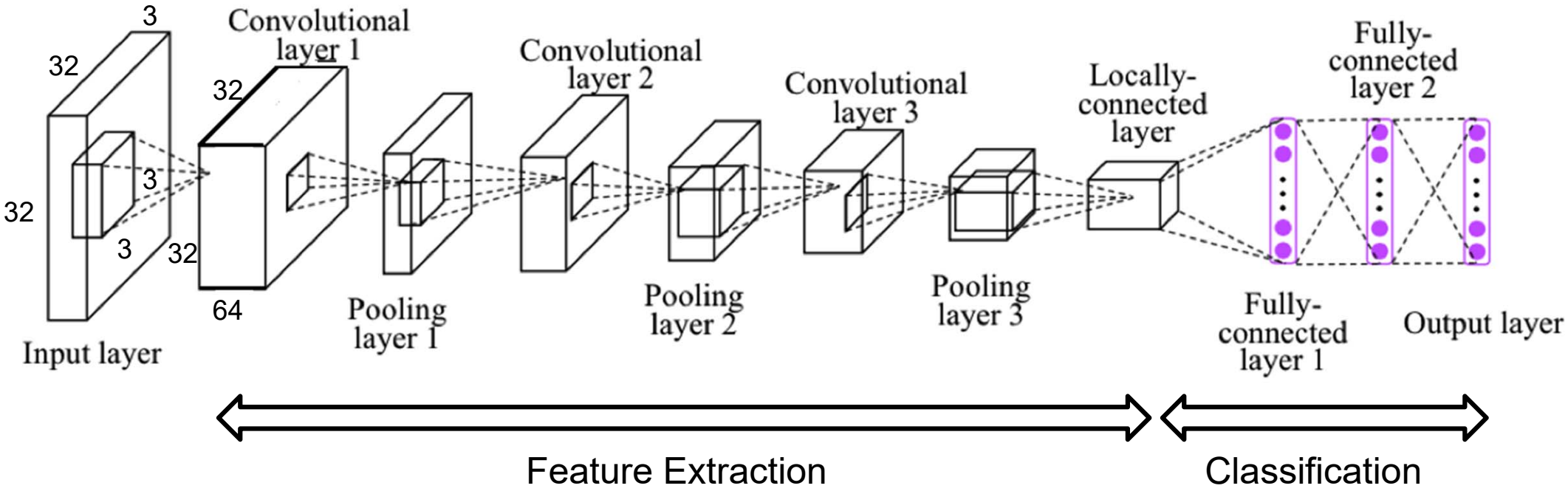
ConvNets : Pooling

- Advantage of Pooling
  - Reducing the number of parameters
  - Generating more robust feature maps: Shift Invariant



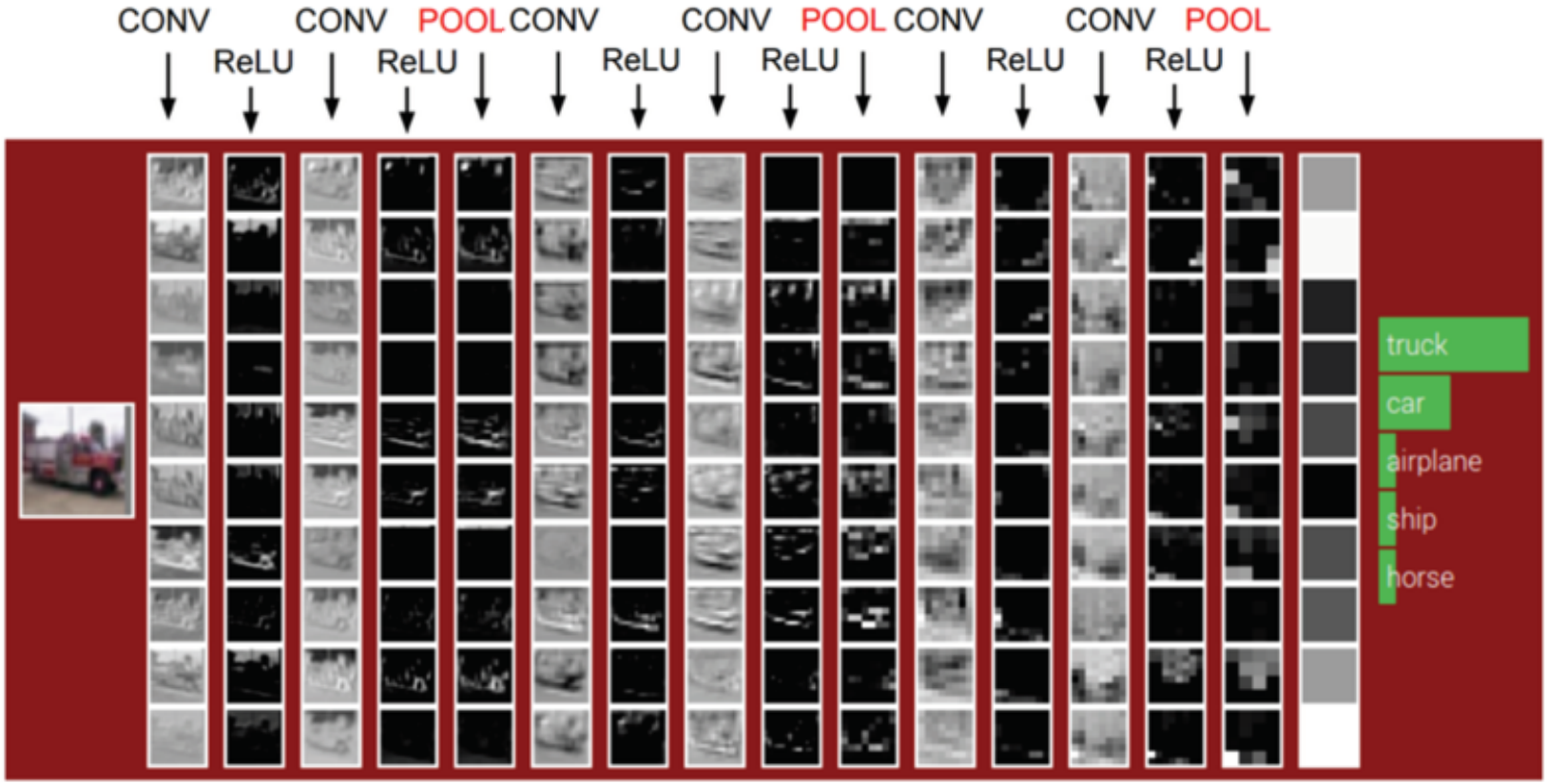
ConvNets

- Graphical Representation



ConvNets

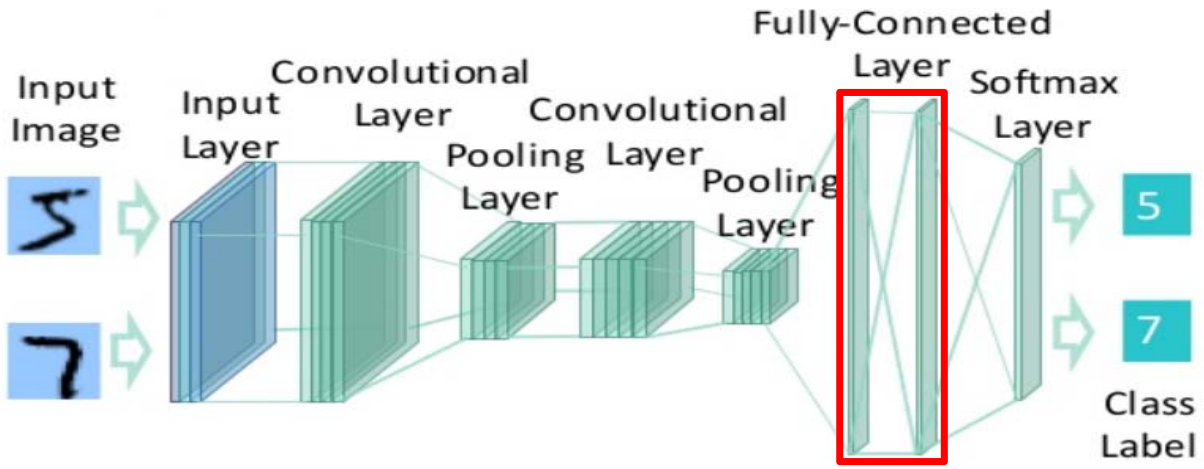
- Example



10x3x3 conv filters, stride 1, pad 1 / 2x2 pool filters , stride 2

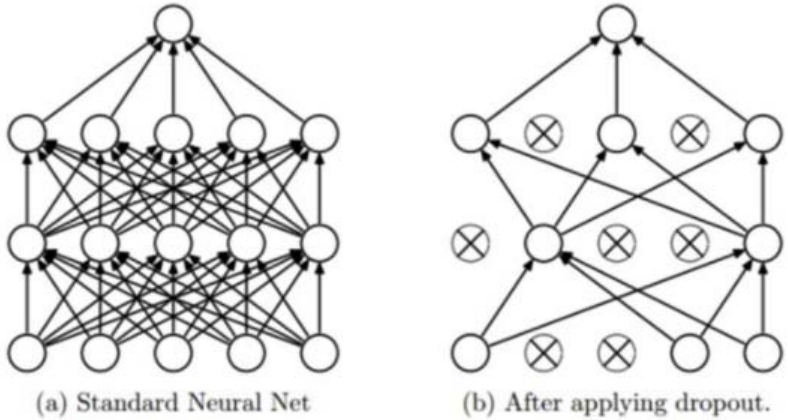
ConvNets : Classification

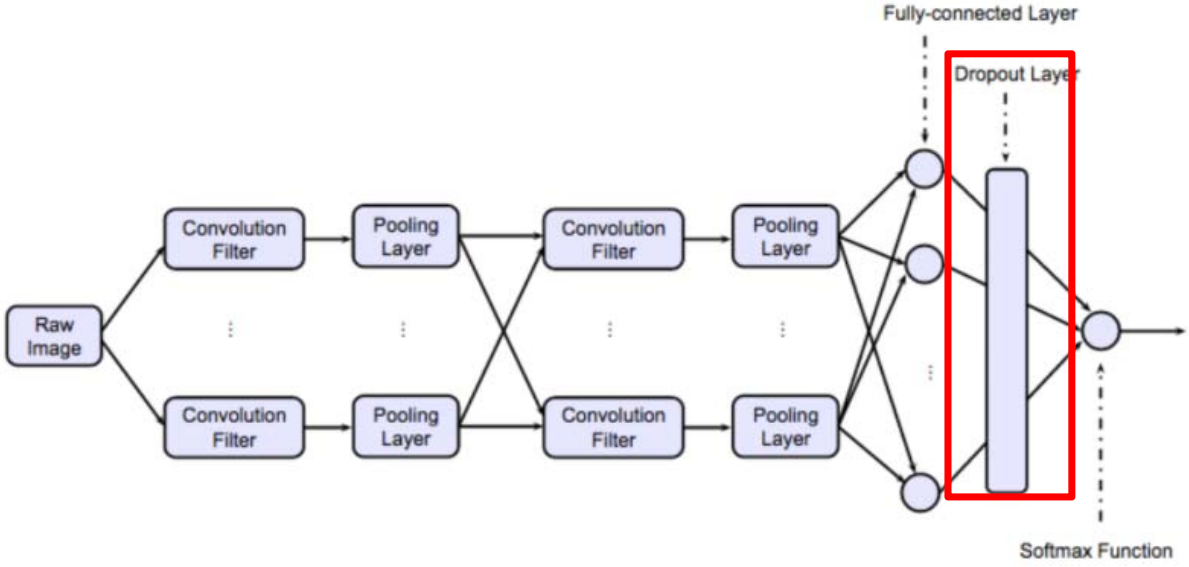
- Fully-connected layer
  - A layer which have full connections to all activations in the previous layer
  - Flatten and connect all the output from the previous layer
  - Cheap way to learn non-linear combinations of the features

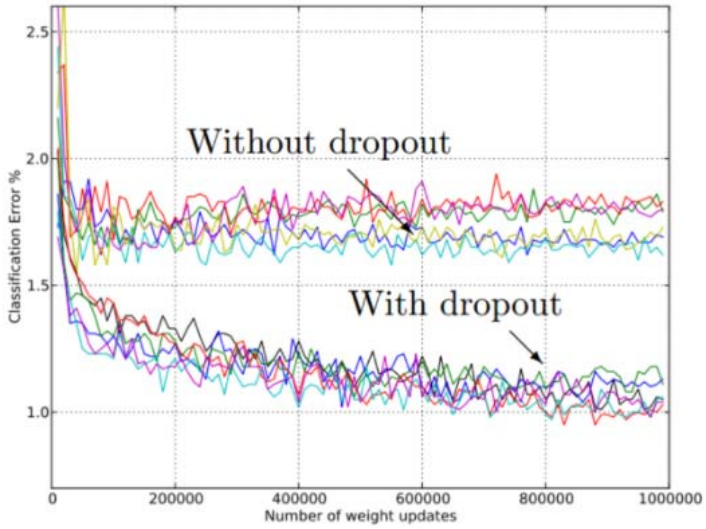


ConvNets : Classification

- Dropout
  - Randomly drop a few neurons not to learn
  - Prevent overfitting

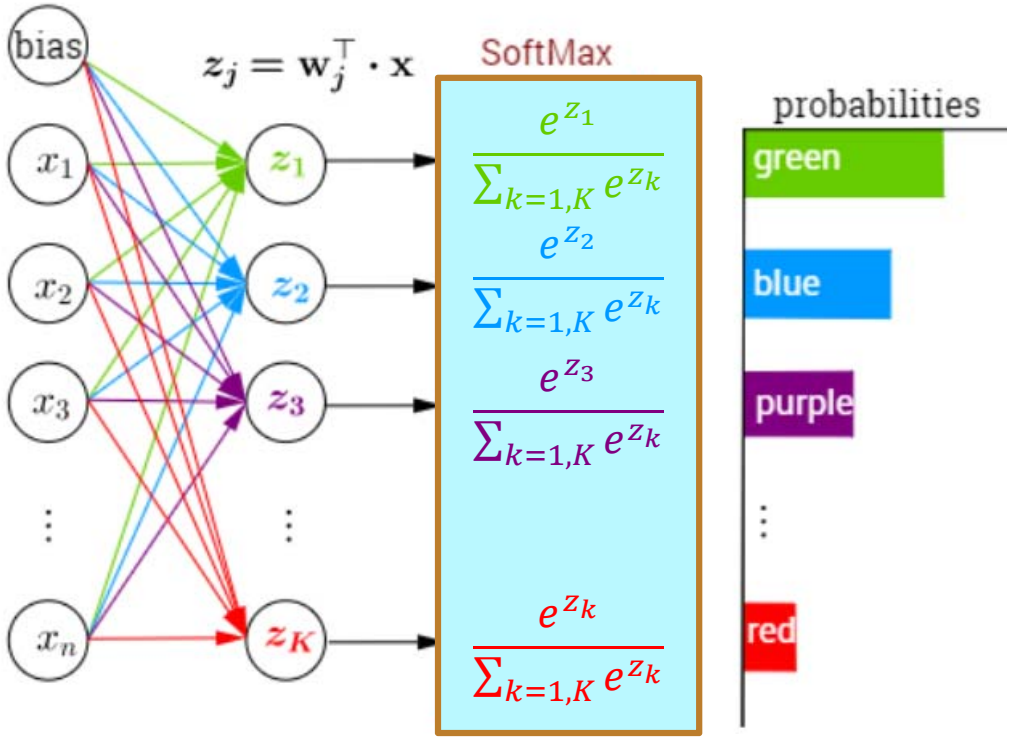






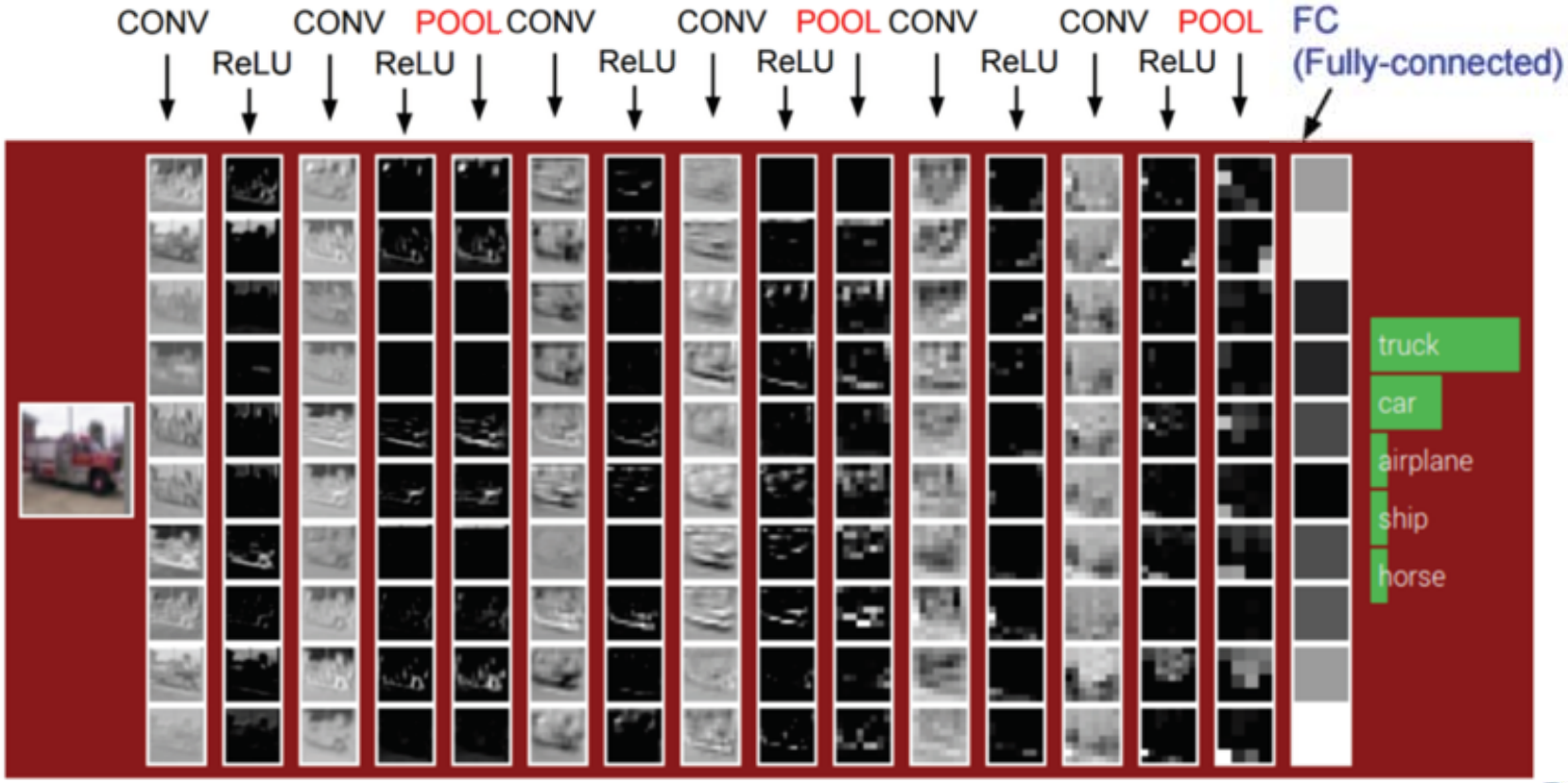
ConvNets : Classification

- Classifier
  - Calculate the probability that the features belong to the classes
  - SoftMax classifier is generally used



ConvNets

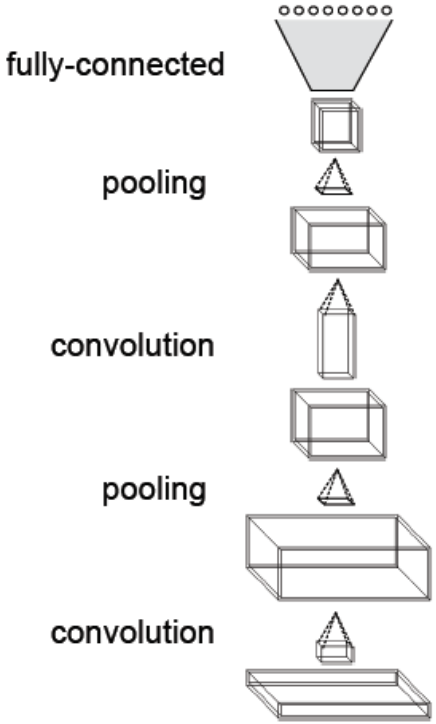
- Example

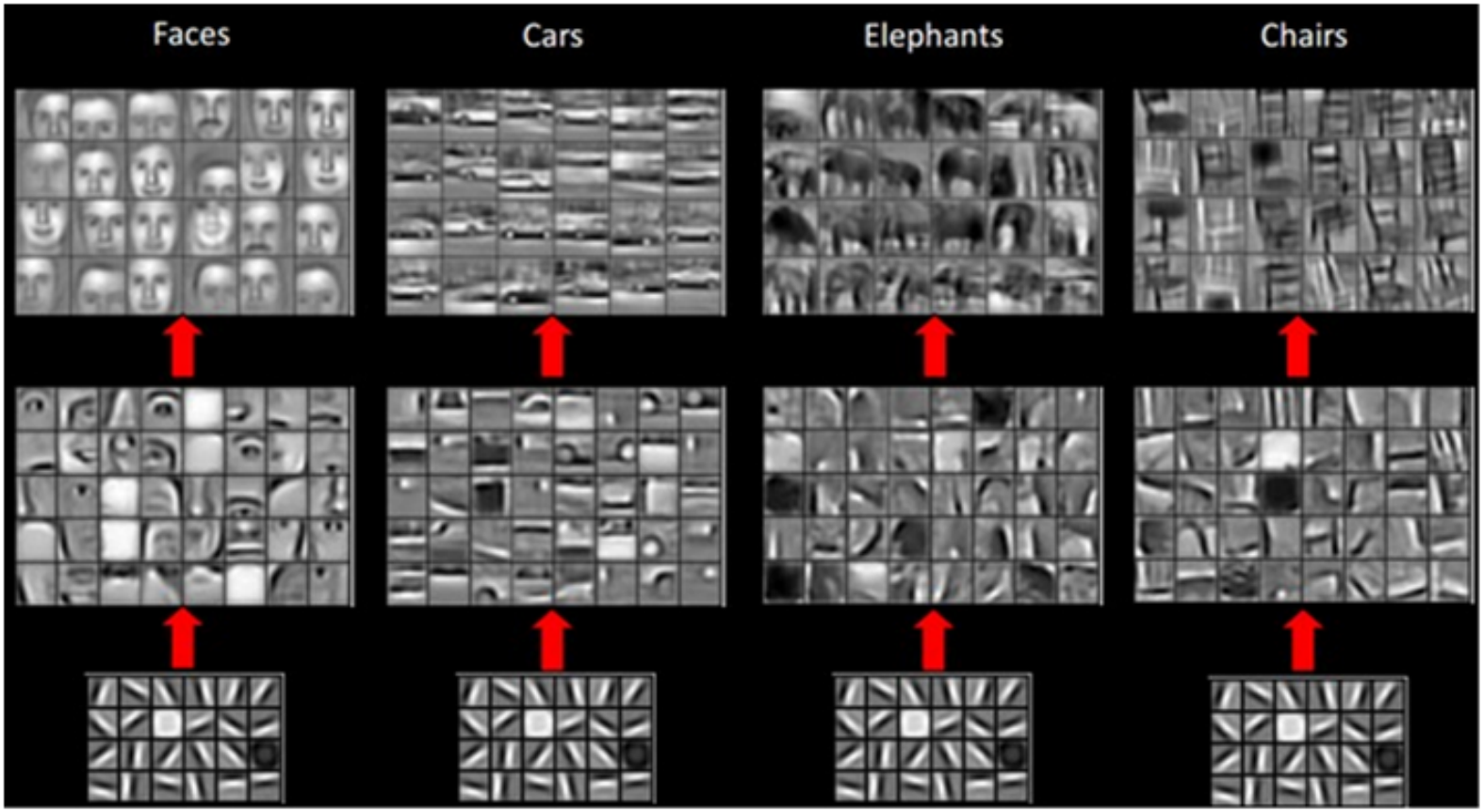


10x3x3 conv filters, stride 1, pad 1 / 2x2 pool filters , stride 2

ConvNets

- Example





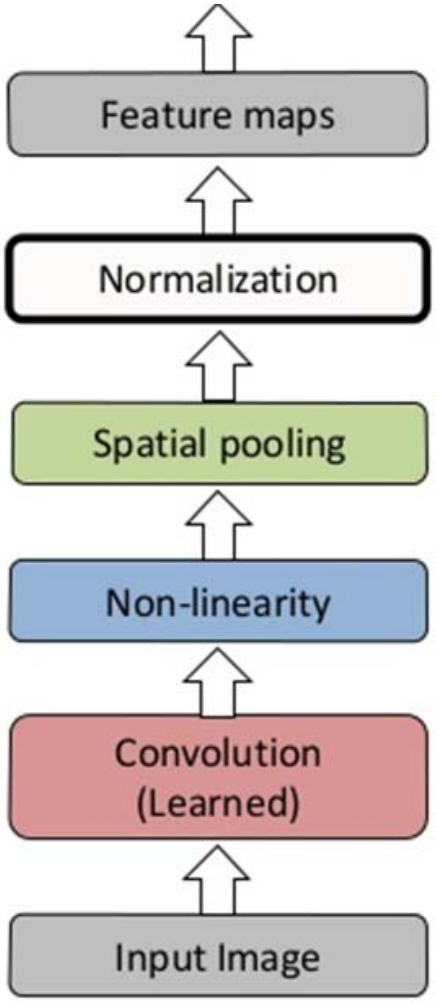
ConvNets

- General Structure

$$f'_{\frac{x}{2},\frac{y}{2}} = max\left(f_{x,y}, f_{x,y+1}, f_{x+1,y}, f_{x+1,y+1}\right)$$

$$f_{x,y} = S\left((I * K)_{xy}\right)$$

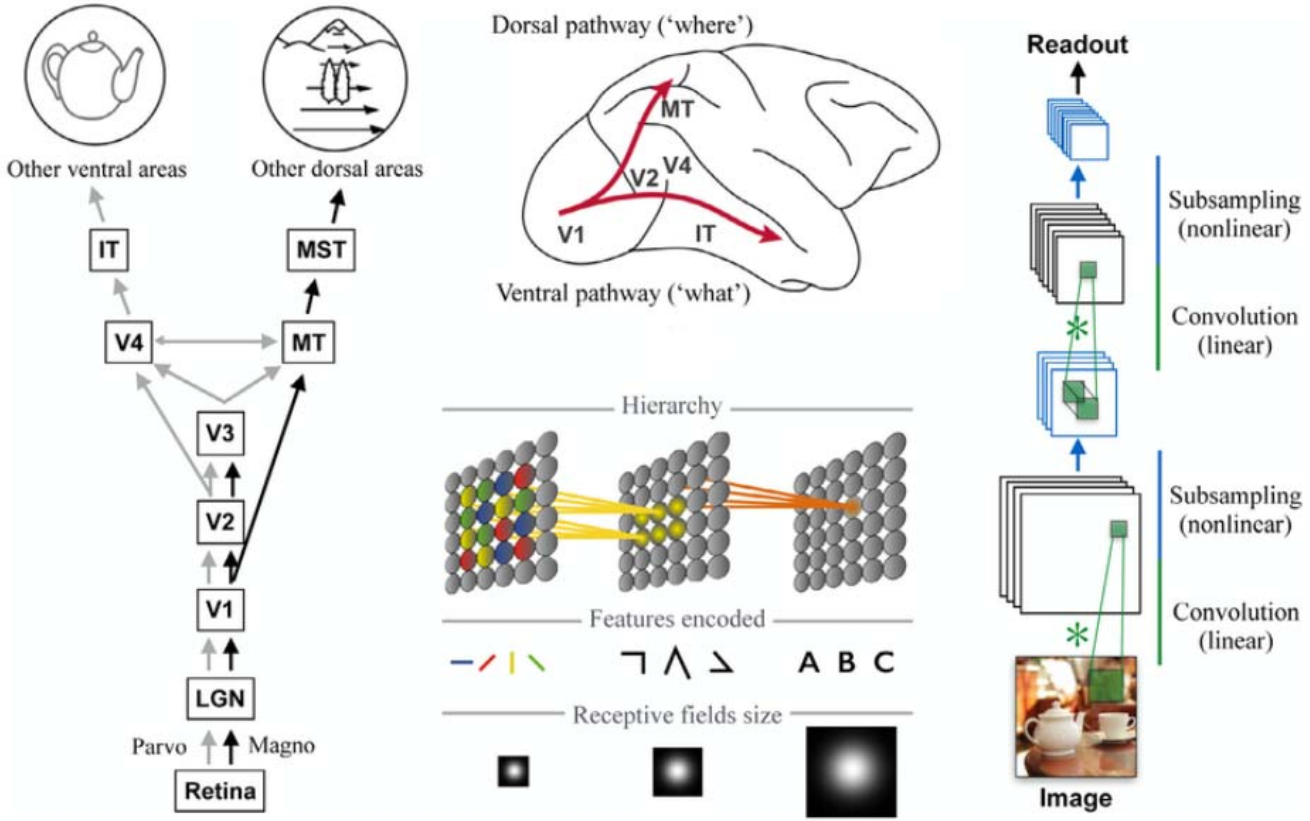
$$(I * K)_{xy} = \sum_{i=1}^{w}\sum_{j=1}^{w} K_{ij} \cdot I_{x+i-1,y+j-1}$$



ConvNets coding (TensorFlow and Keras)
https://youtu.be/j-3vuBynnOE
https://youtu.be/WvoLTXIjBYU

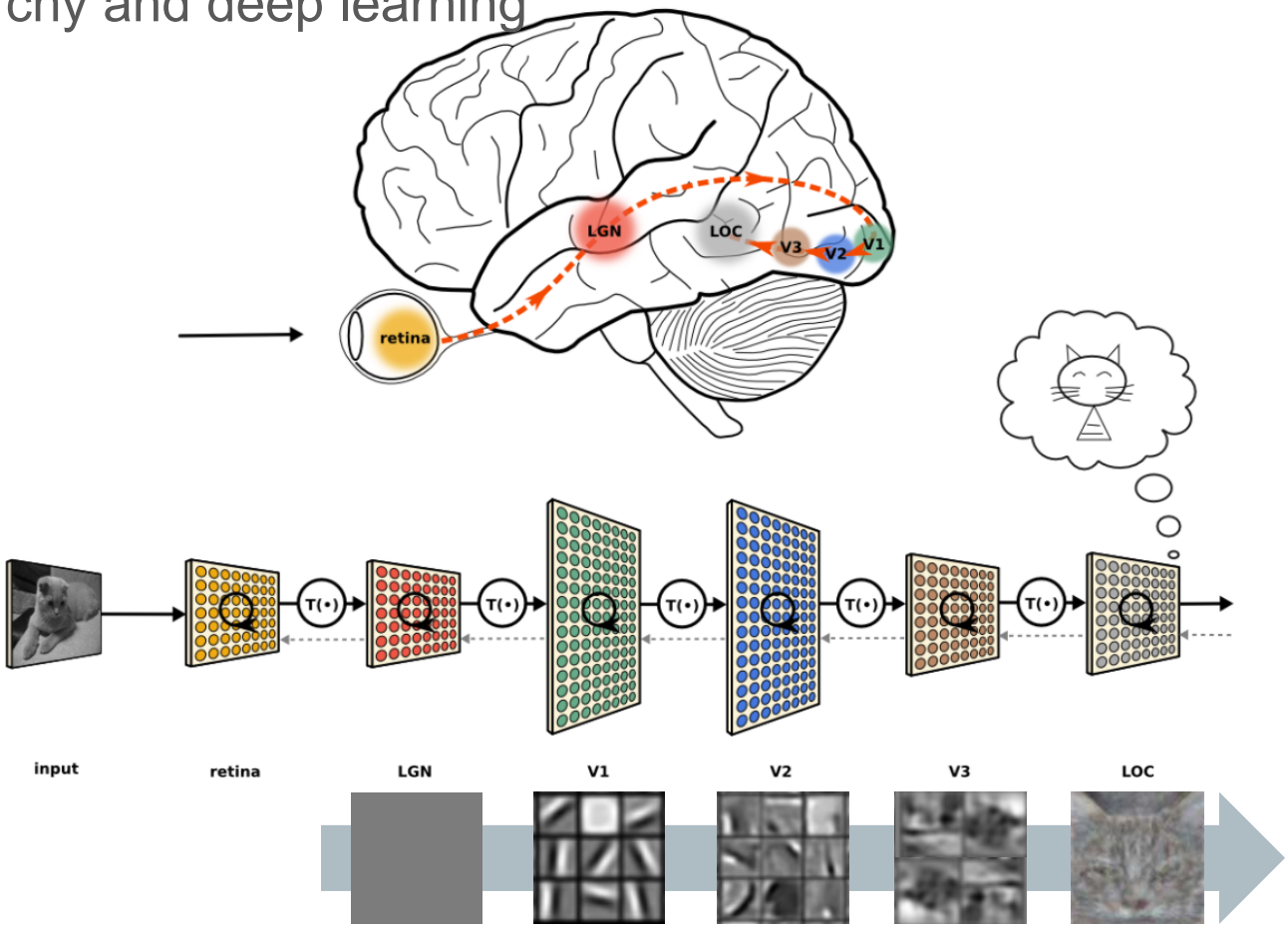
Neural network applications in neuroscience

- Visual hierarchy and deep learning



Neural network applications in neuroscience

- Visual hierarchy and deep learning

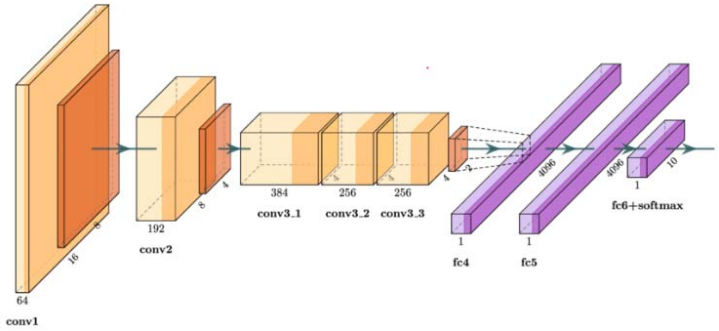


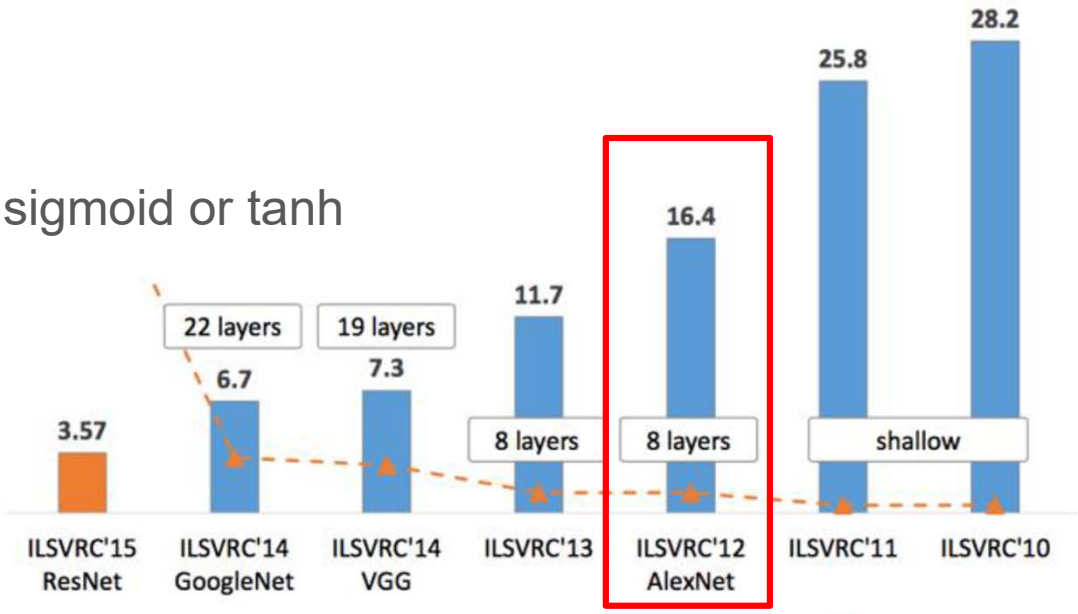
AlexNet / ResNet

AlexNet

ImageNet Classification with Deep Convolutional Neural Networks (NIPS 2012)

- Implemented a convolutional neural network
  - Use Deep Learning
  - Open source
  - Use GPU
- Further theoretical improvements
  - Rectified Linear Units (ReLU) instead of sigmoid or tanh
  - Dropout
  - Overlapping Pooling
  - Data augmentation





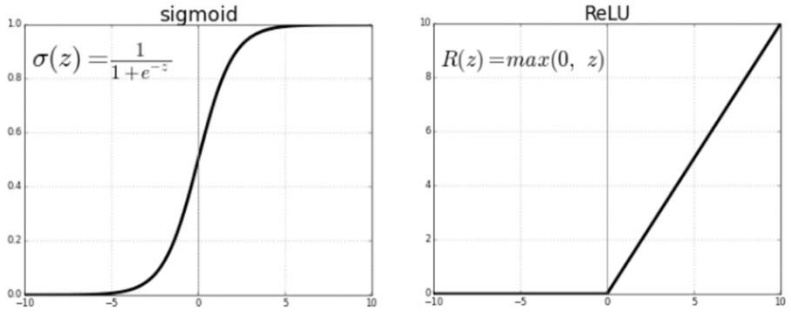
AlexNet

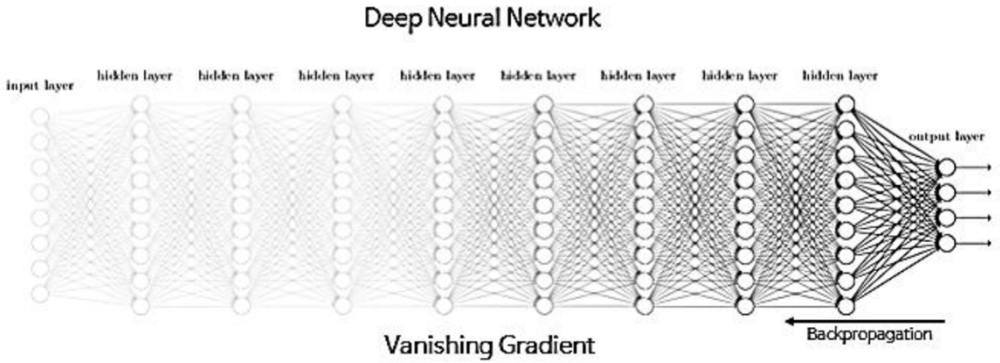
- **Rectified Linear Units (ReLUs)**
  - Vanishing Gradient Problem
  - The standard way

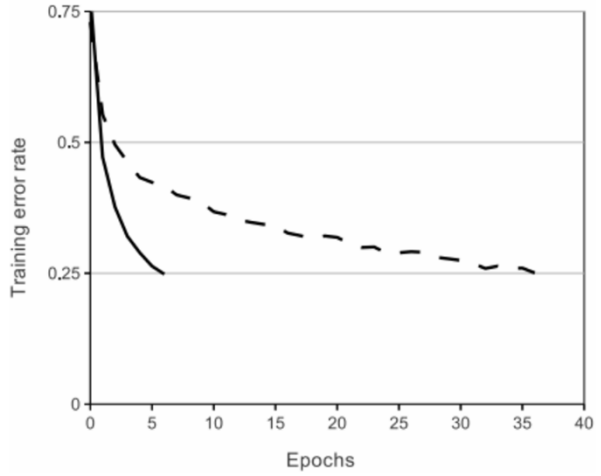
$$f(x) = \tanh(x) \; or \; f(x) = (1 + e^{-x})^{-1}$$

  - Rectified Linear Units (ReLUs)

$$f(x) = \max(0, x)$$

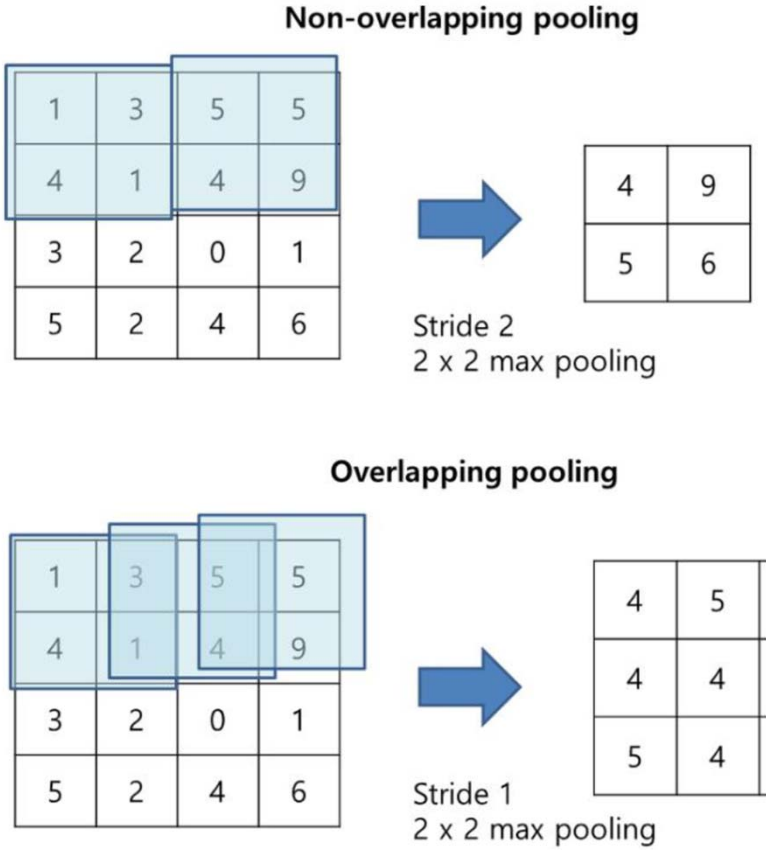






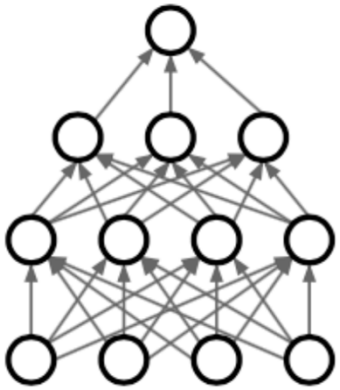
AlexNet

- Overlapping pooling
  - Traditionally, the neighborhoods summarized by adjacent pooling units do not overlap
  - Reduces the top-1 and top-5 error rates by 0.4% and 0.3%, respectively

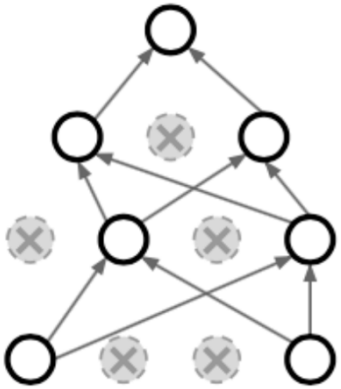


AlexNet

- Use **dropout** in the first two fully connected layers
  - For each forward pass, randomly drop some neurons to zero
  - Probability of dropping is a hyperparameter (0.5 most common)
  - Forces the network to have a redundant representation
  - Prevents co-adaptation of features



Without Dropout



With Dropout



AlexNet

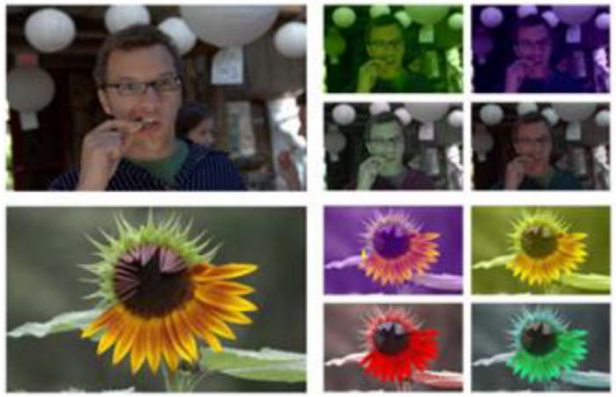
- Data Augmentation
  - 60 million parameters, 1000 classes of ILSVRC ⇒ insufficient to learn so many parameters without considerable overfitting
  - To artificially enlarge the dataset using label preserving transformations
  - Generating image translation and horizontal reflections



  - Altering the intensities of the RGB channels in training images

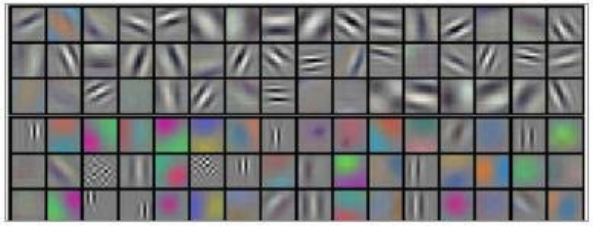
$$I_{xy} = [I_{xy}^R,\ I_{xy}^G,\ I_{xy}^B]^T + [\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3][\alpha_1\lambda_1, \alpha_2\lambda_2, \alpha_3\lambda_3]^T$$

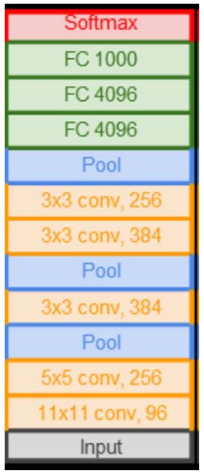
$$\alpha_i \sim N(0,\ 0.1)$$

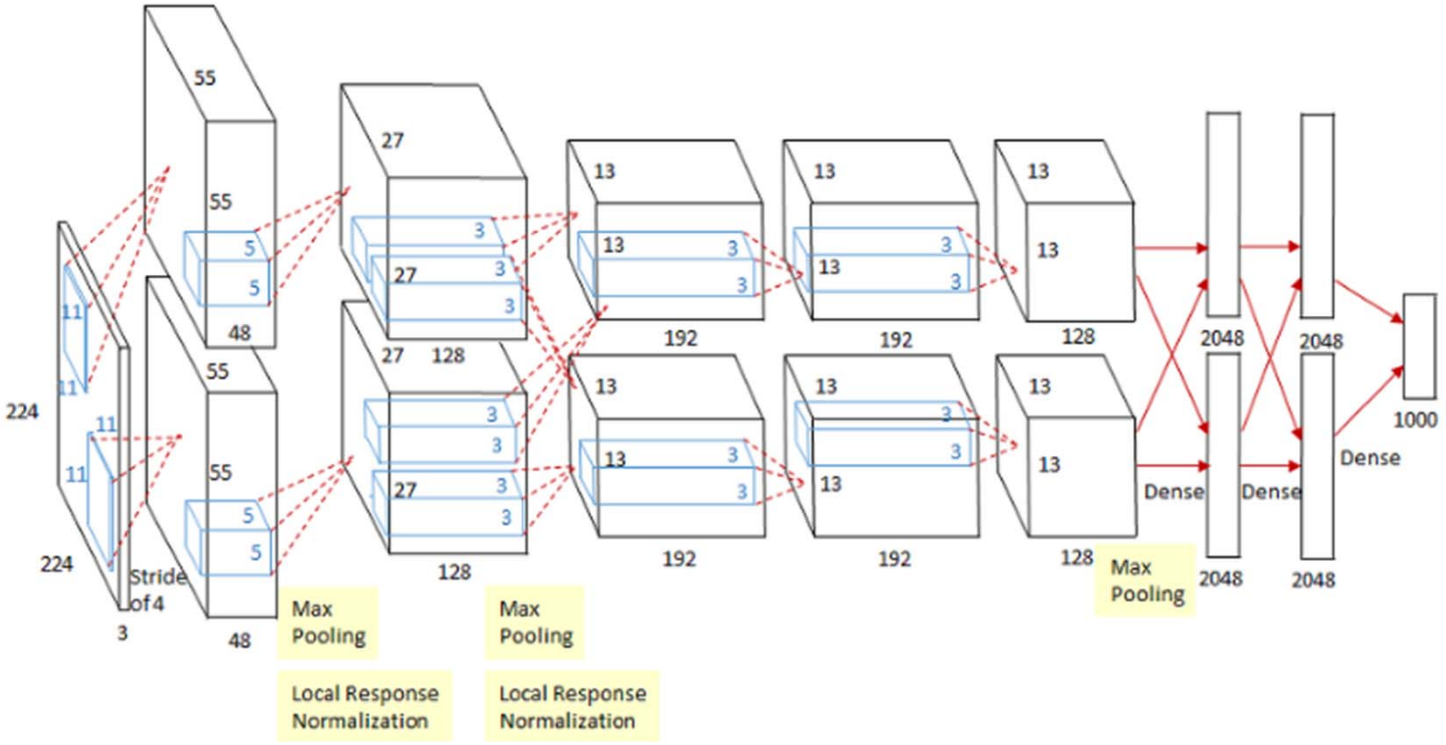


AlexNet

- 5 Convolutional Layer & 3 Fully-connected Layer
- Down-sampled the images to a fixed resolution of 256 ×256







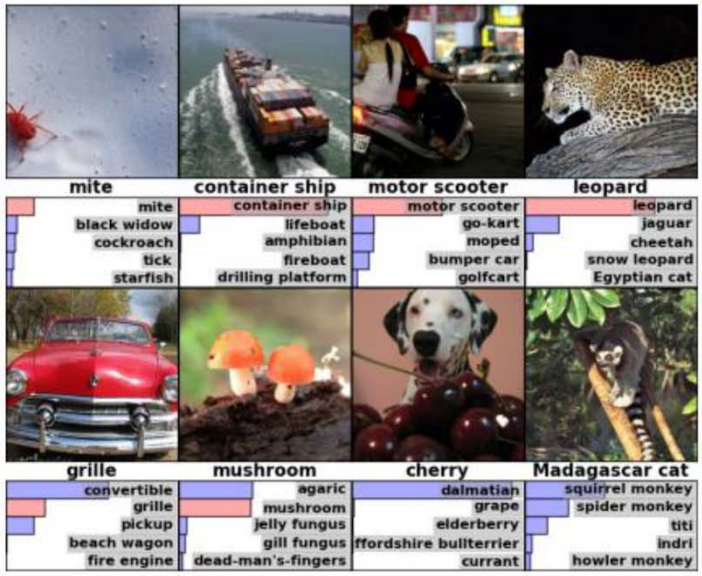
AlexNet

- Results
  - ILSVRC 2010

Model	Top-1	Top-5
Sparse coding [2]	*47.1%*	*28.2%*
SIFT + FVs [24]	*45.7%*	*25.7%*
CNN	**37.5%**	**17.0%**

ILSVRC 2012

Model	Top-1 (val)	Top-5 (val)	Top-5 (test)
SIFT + FVs [7]	—	—	*26.2%*
1 CNN	40.7%	18.2%	—
5 CNNs	38.1%	16.4%	**16.4%**
1 CNN*	39.0%	16.6%	—
7 CNNs*	36.7%	15.4%	**15.3%**

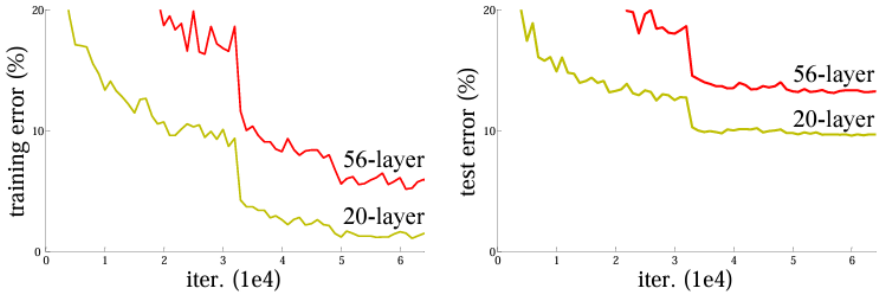




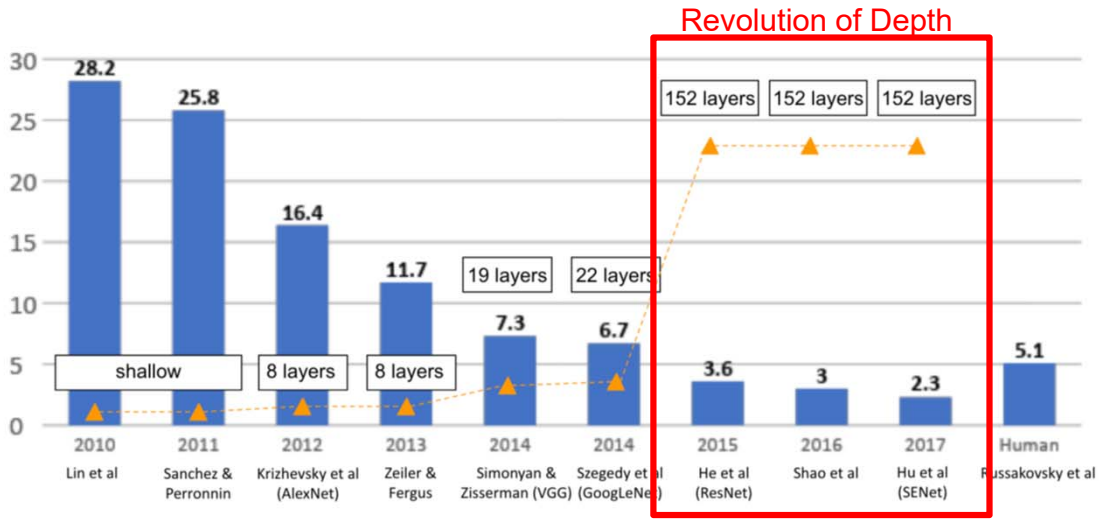
ResNet

Deep Residual Learning for Image Recognition (CVPR 2015)

- Many visual recognition tasks have greatly benefited from very deep models
- But, the deeper network has higher training error, and thus test error



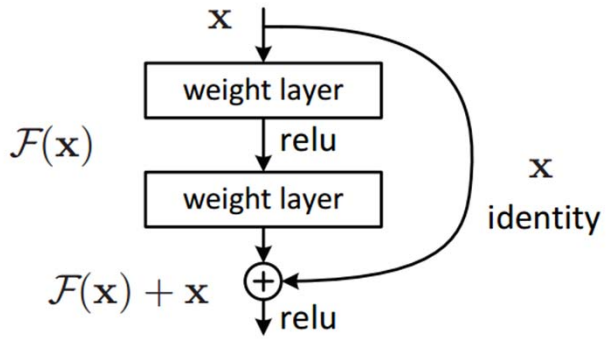
[Degradation Problem]
Training error (left) and test error (right) on CIFAR-10



ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners

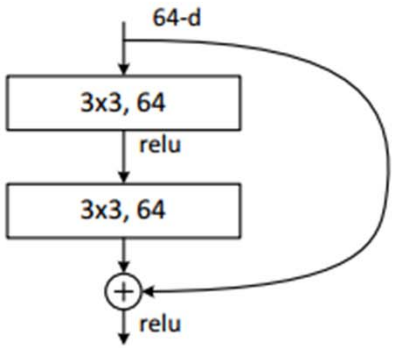
ResNet

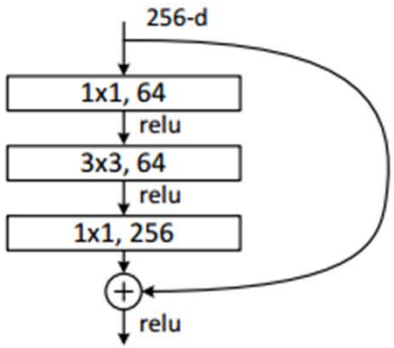
- Skip / Shortcut Connection in Residual Network(ResNet)



<Residual learning: a building block>

Bottleneck Design



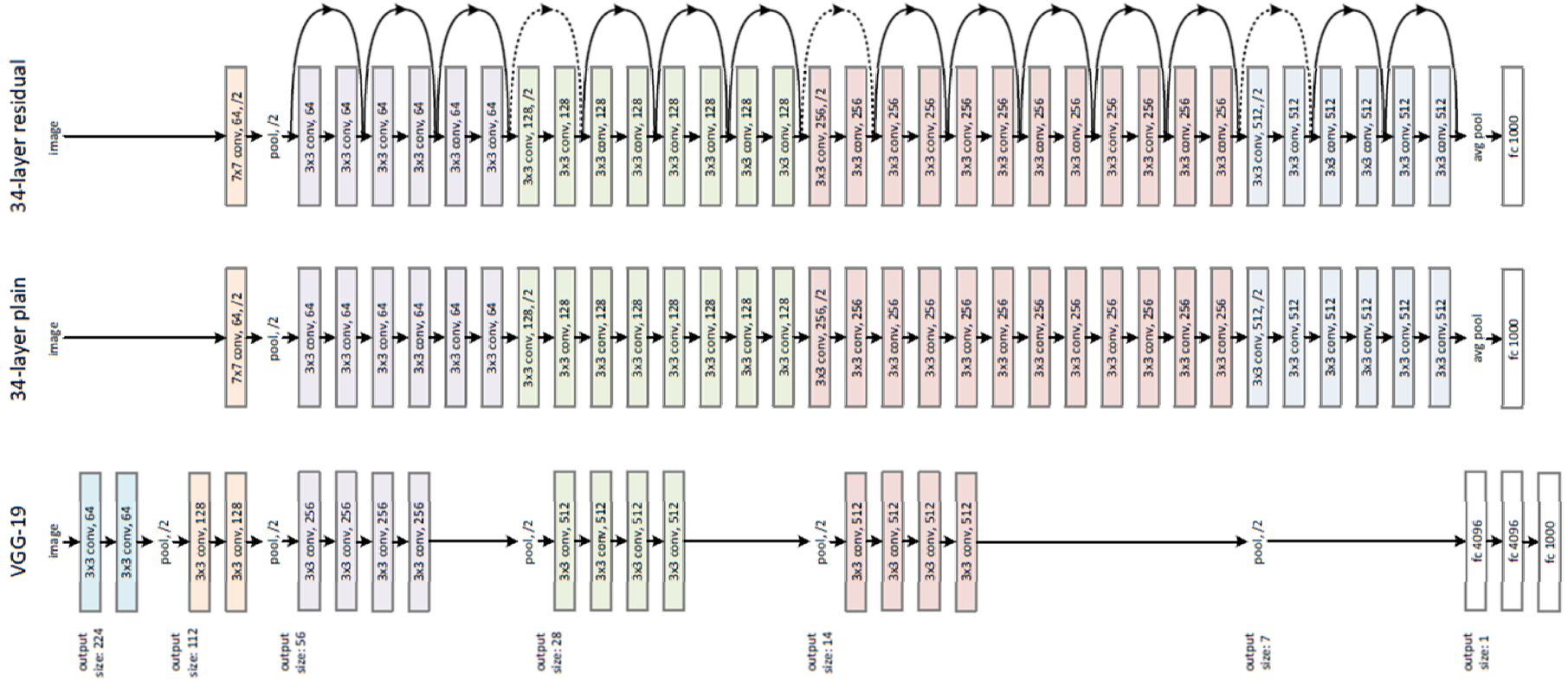


Adeeper residual functionFfor ImageNet.
Left: building block (on 56×56 feature maps) for ResNet34
Right: “bottleneck” building block for ResNet-50/101/152

ResNet

- ResNet Architecture

Skip connection : *The great gradient highway!!*



Vision Transformer (ViT)

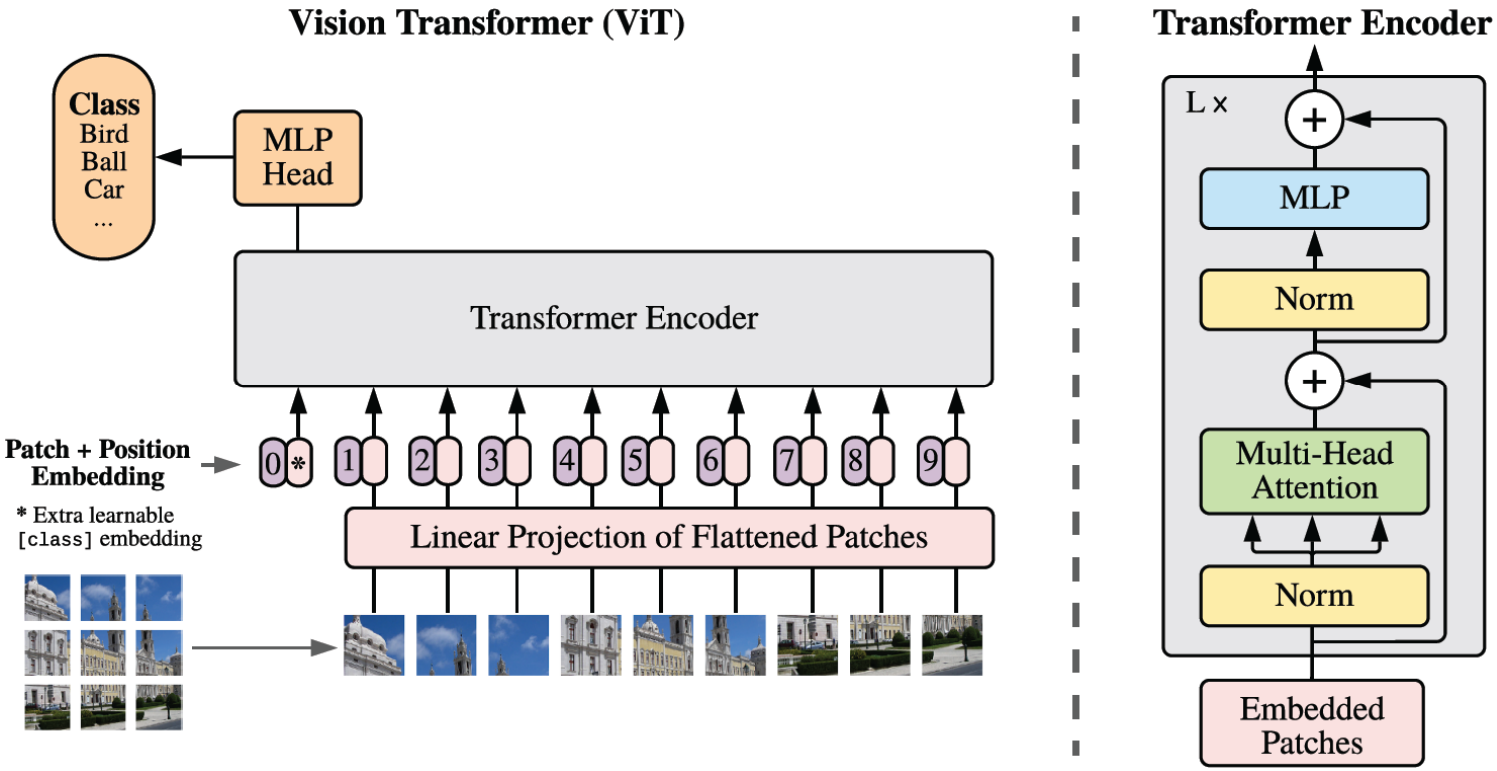
Vision Transformer (ViT)

- The Vision Transformer (ViT) model was introduced in 2021 in a conference research paper titled "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," published at ICLR 2021.
- Idea is to create a Vision Transformer using the **Transformer** encoder architecture, with the fewest possible modifications, and apply it to image classification tasks.
- The fine-tuning code and pre-trained ViT models are accessible on Google Research's GitHub. The ViT models were pre-trained on the ImageNet and ImageNet-21k datasets.
- ViT has much less **image-specific inductive bias** than CNNs
- ViT models outperform the current state-of-the-art (CNN) by almost x4 in terms of computational efficiency and accuracy.

Paper : An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale (ICLR 2021)

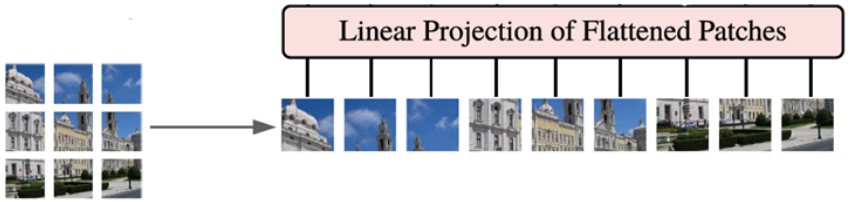
Vision Transformer (ViT)

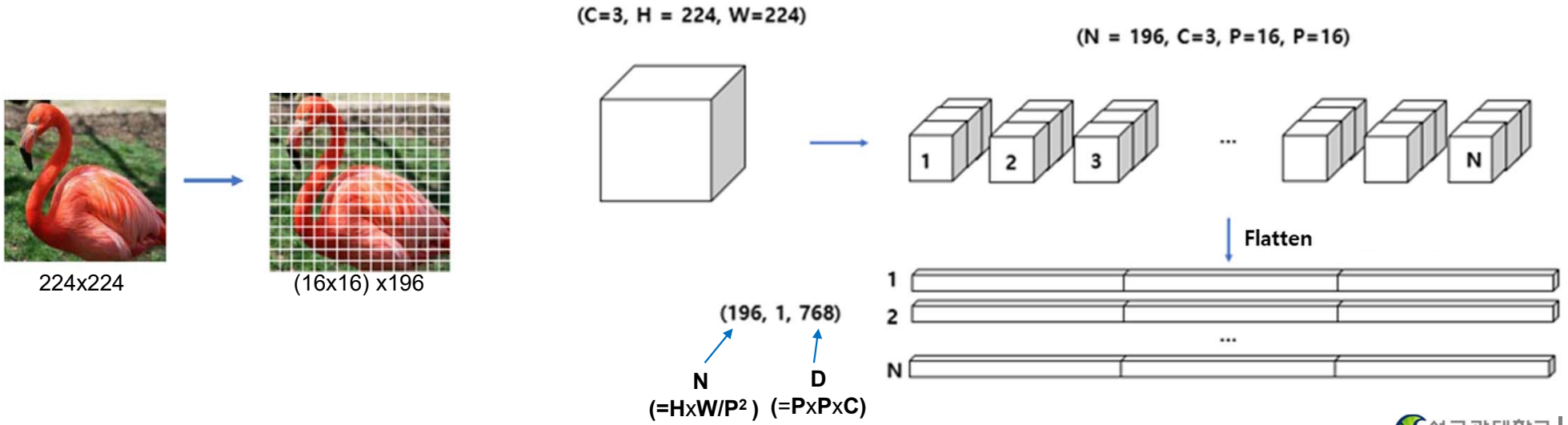
- Model overview



Vision Transformer (ViT)

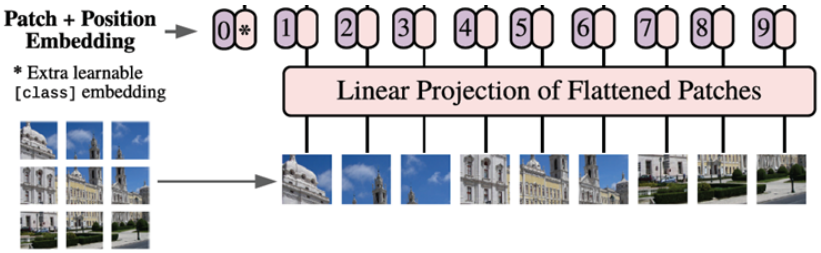
- Linear Projection of Flatten Patches
  - The input image is divided into fixed-size square patches.
  - Each patch is then linearly transformed into a vector using a learnable linear projection.
  - This results in a sequence of patch embeddings, which serve as the input tokens for the subsequent layers.

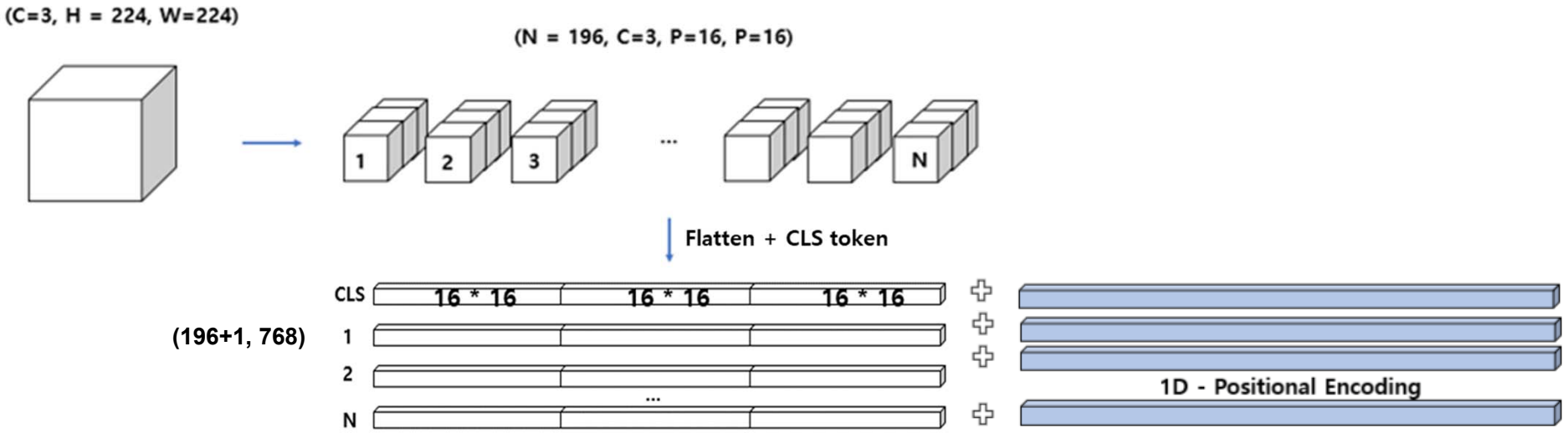




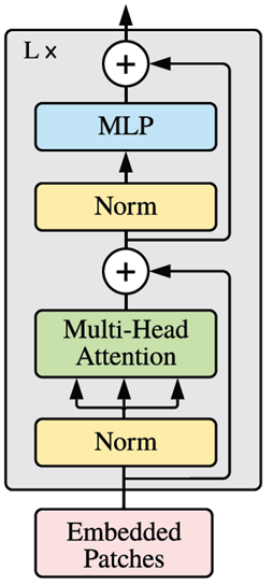
Vision Transformer (ViT)

- Positional & Class Embedding
  - Since the Vision Transformer lacks any inherent understanding of spatial relationships, positional information needs to be explicitly provided.
  - Also, similar to BERT's [class] token, it prepends a learnable embedding to the sequence of embedded patches.

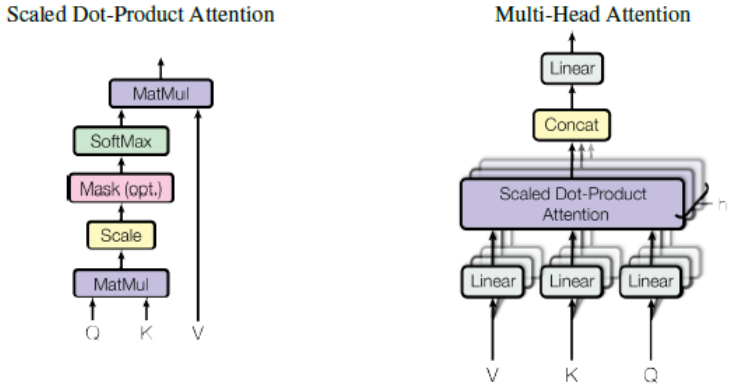




Vision Transformer (ViT)



- Multi-Head Self-Attention
  - The self-attention mechanism captures the relationships between different patches in the input sequence.
  - This mechanism allows the model to focus on important patches while considering both local and global contexts.
  - Multi-head attention employs multiple sets of learnable parameters (attention heads) to capture different types of relationships.



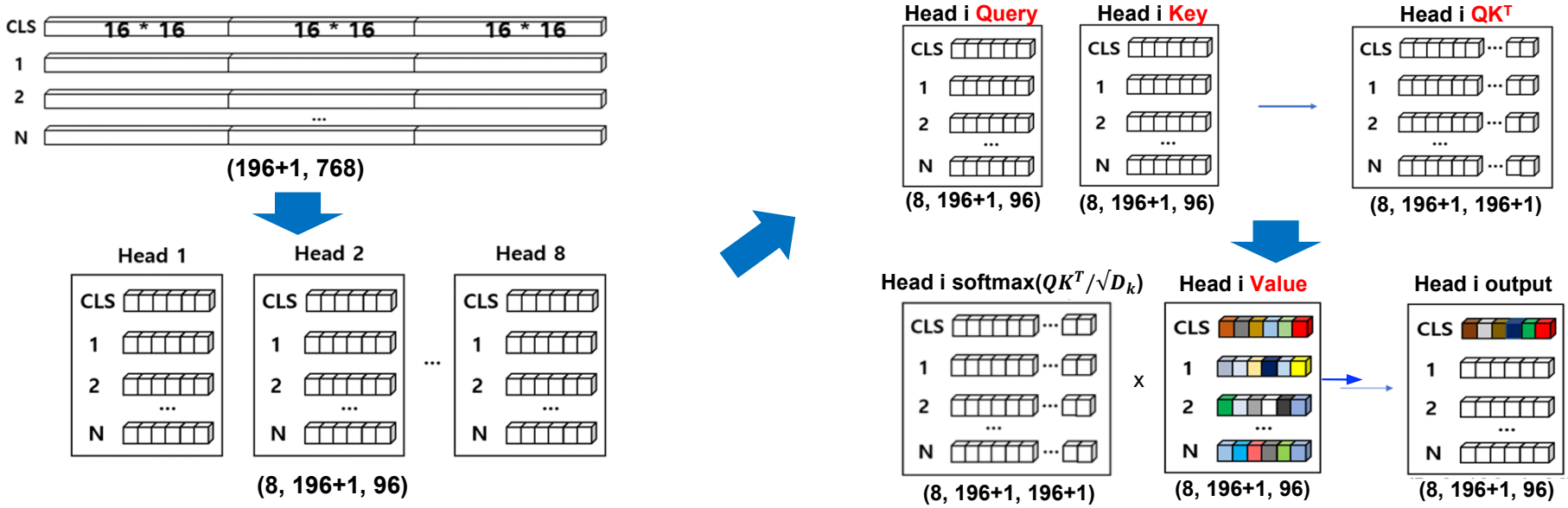
<Attention mechanism of Transformer>

Vision Transformer (ViT)

- Multi-Head Self-Attention
  - The attention mechanism used in the Transformer uses three variables: Q (Query), K(Key), and V (Value).
  - It calculates the association (attention weight) between the Query token and the Key token and multiplies the Value associated with each Key.

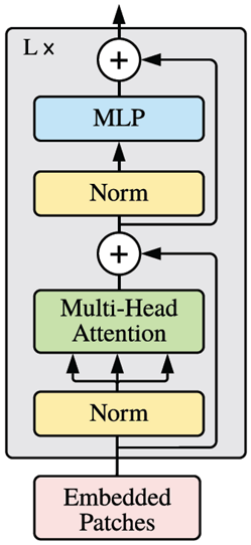
$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V = \text{Attention Value Matrix } a$$

<ex: Assume the number of Head are 8>

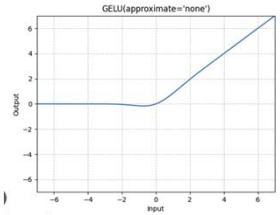


Vision Transformer (ViT)

- Transformer Encoder



- **Multi-Head Self Attention Layer** (MSP): This layer concatenates all the attention outputs linearly to the right dimensions. The many attention heads help train local and global dependencies in an image.
- **Multi-Layer Perceptrons** (MLP) Layer: This layer contains a two-layer with Gaussian Error Linear Unit (GELU).



- **Layer Normalization** (LN): This is added prior to each block as it does not include any new dependencies between the training images. This thereby helps improve the training time and overall performance.

$$\mathbf{z}_0 = [\mathbf{x}_{\text{class}};\ \mathbf{x}_p^1\mathbf{E};\ \mathbf{x}_p^2\mathbf{E};\cdots;\ \mathbf{x}_p^N\mathbf{E}] + \mathbf{E}_{pos}, \qquad \mathbf{E} \in \mathbb{R}^{(P^2\cdot C)\times D},\ \mathbf{E}_{pos} \in \mathbb{R}^{(N+1)\times D} \tag{1}$$

$$\mathbf{z}'_\ell = \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}, \qquad \ell = 1 \ldots L \tag{2}$$

$$\mathbf{z}_\ell = \text{MLP}(\text{LN}(\mathbf{z}'_\ell)) + \mathbf{z}'_\ell, \qquad \ell = 1 \ldots L \tag{3}$$

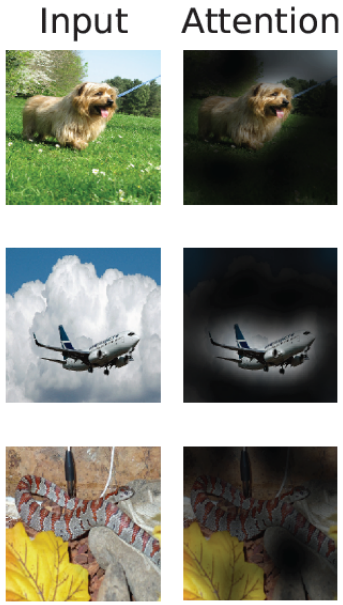
$$\mathbf{y} = \text{LN}(\mathbf{z}_L^0) \tag{4}$$

Vision Transformer (ViT)

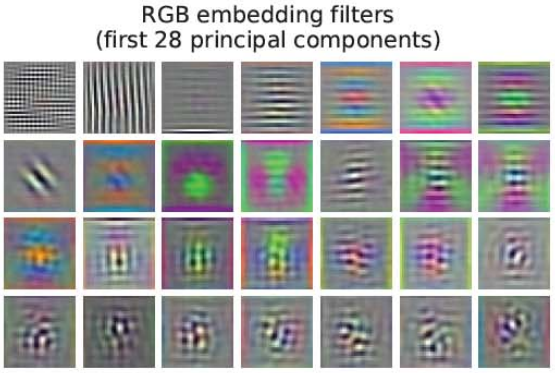
- Total architecture
  1) Split an image into patches
  2) Flatten the patches
  3) Produce lower-dimensional linear embeddings from the flattened patches
  4) Add positional embeddings
  5) Feed the sequence as an input to a standard transformer encoder
  6) Pretrain the model with image labels (fully supervised on a huge dataset)
  7) Finetune on the downstream dataset for image classification

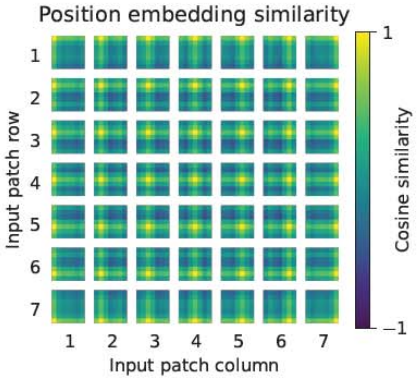
Vision Transformer (ViT)

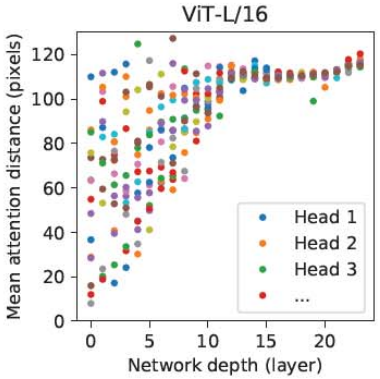
- Inspecting ViT



Representative examples of attention from the output token to the input space







(Left) Filters of the initial linear embedding of RGB values of ViT-L/32.
(Center) Similarity of position embeddings of ViT-L/32.
(Right) Size of attended area by head and network depth

Vision Transformer (ViT)

- Applications of Vision Transformers
  - ***Image Classification***: Classifying images into predefined categories
  - ***Object Detection***: Identifying and localizing objects within images
  - ***Semantic Segmentation***: Assigning a label to each pixel in an image to identify object boundaries
  - ***Image Generation***: Generating new images based on a given context or description
- Limitations
  - ***Large Datasets***: Training Vision Transformers effectively often requires large datasets, which might not be available for all domains.
  - ***Computational Demands***: Training ViT can be computationally intensive due to the self-attention mechanisms.
  - ***Spatial Information***: ViT's sequential processing might not capture fine-grained spatial patterns as effectively as CNNs for tasks like segmentation.

Vision Transformer (ViT)

- ViT vs. Convolutional Neural Networks

Do Vision Transformers See Like Convolutional Neural Networks?
https://arxiv.org/abs/2108.08810

- ***Inductive bias***: It has much less image-specific inductive bias(assumptions that the learner uses to predict outputs of given inputs that it has not encountered)
- ***Input Representation***: While CNNs process raw pixel values directly, ViT divides the input image into patches and transforms them into tokens.
- ***Processing Mechanism***: CNNs use convolutional and pooling layers to hierarchically capture features at different spatial scales. ViT employs self-attention mechanisms to consider relationships among all patches.
- ***Global Context***: ViT inherently captures global context through self-attention, which helps in recognizing relationships between distant patches. CNNs rely on pooling layers for coarse global information.
- ***Data Efficiency***: CNNs often require large amounts of labeled data for training, whereas ViT can benefit from pre-training on large datasets and then fine-tuning on specific tasks.

Any Questions?

Thanks!